EC6501
DIGITAL COMMUNICATION

UNIT I SAMPLING & QUANTIZATION

SAMPLING:

Sampling Theorem for strictly band-limited signals

1. a signal which is limited to $-W < f < W$, can be completely described by $\left\{ g\left(\frac{n}{2W} \right) \right\}$.

2. The signal can be completely recovered from $\left\{ g\left(\frac{n}{2W} \right) \right\}$

Nyquist rate = $2W$

Nyquist interval = $1/2W$

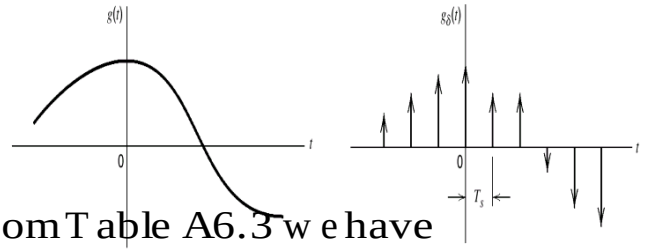
When the signal is not band-limited (under sampling) aliasing occurs. To avoid aliasing, we may limit the signal bandwidth or have higher sampling rate.

Let $g_\delta(t)$ denote the ideal sampled signal

$$g_\delta(t) = \sum_{n=-\infty}^{\infty} g(nT_s)\,\delta(t - nT_s) \qquad (3.1)$$

where T_s : sampling period

$f_s = 1/T_s$: sampling rate



(a) (b)

From Table A6.3 we have

$$g(t)\sum_{n=-\infty}^{\infty}\delta(t-nT_s) \Leftrightarrow$$

$$G(f) * \frac{1}{T_s}\sum_{m=-\infty}^{\infty}\delta\left(f-\frac{m}{T_s}\right)$$

$$= \sum_{m=-\infty}^{\infty} f_s G(f - mf_s)$$

$$g_\delta(t) \Leftrightarrow f_s \sum_{m=-\infty}^{\infty} G(f - mf_s) \qquad (3.2)$$

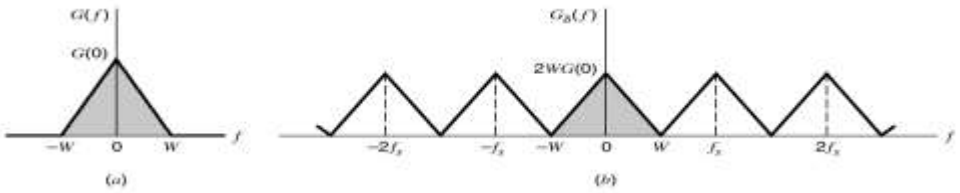
or we may apply Fourier Transform on (3.1) to obtain

$$G_\delta(f) = \sum_{n=-\infty}^{\infty} g(nT_s)\exp(-j2\pi\, nf\, T_s) \qquad (3.3)$$

or $$G_\delta(f) = f_s G(f) + f_s \sum_{\substack{m=-\infty \\ m\neq 0}}^{\infty} G(f - mf_s) \qquad (3.5)$$

If $G(f) = 0$ for $|f| \geq W$ and $T_s = 1/2W$

$$G_\delta(f) = \sum_{n=-\infty}^{\infty} g\left(\frac{n}{2W}\right)\exp\left(-\frac{j\pi\, n\, f}{W}\right) \qquad (3.4)$$



With

1. $G(f) = 0$ for $|f| \geq W$

2. $f_s = 2W$

we find from Equation (3.5) that

$$G(f) = \frac{1}{2W} G_\delta(f) \ , \ -W < f < W \qquad (3.6)$$

Substituting (3.4) into (3.6) we may rewrite $G(f)$ as

$$G(f) = \frac{1}{2W} \sum_{n=-\infty}^{\infty} g(\frac{n}{2W}) \exp(-\frac{j\pi nf}{W}) , -W < f < W \quad (3.7)$$

$g(t)$ is uniquely determined by $g(\frac{n}{2W})$ for $-\infty < n < \infty$

or $\{ g(\frac{n}{2W}) \}$ contains all information of $g(t)$

To reconstruct $g(t)$ from $\{ g(\frac{n}{2W}) \}$, we may have

$$g(t) = \int_{-\infty}^{\infty} G(f) \exp(j2\pi ft) df$$

$$= \int_{-W}^{W} \frac{1}{2W} \sum_{n=-\infty}^{\infty} g(\frac{n}{2W}) \exp(-\frac{j\pi n f}{W}) \exp(j2\pi f t) df$$

$$= \sum_{n=-\infty}^{\infty} g(\frac{n}{2W}) \frac{1}{2W} \int_{-W}^{W} \exp\left[j2\pi f (t - \frac{n}{2W}) \right] df \quad (3.8)$$

$$= \sum_{n=-\infty}^{\infty} g(\frac{n}{2W}) \frac{\sin(2\pi Wt - n\pi)}{2\pi Wt - n\pi}$$

$$= \sum_{n=-\infty}^{\infty} g(\frac{n}{2W}) \sin c(2Wt - n) \ , \ -\infty < t < \infty \qquad (3.9)$$

(3.9) is an interpolation formula of $g(t)$

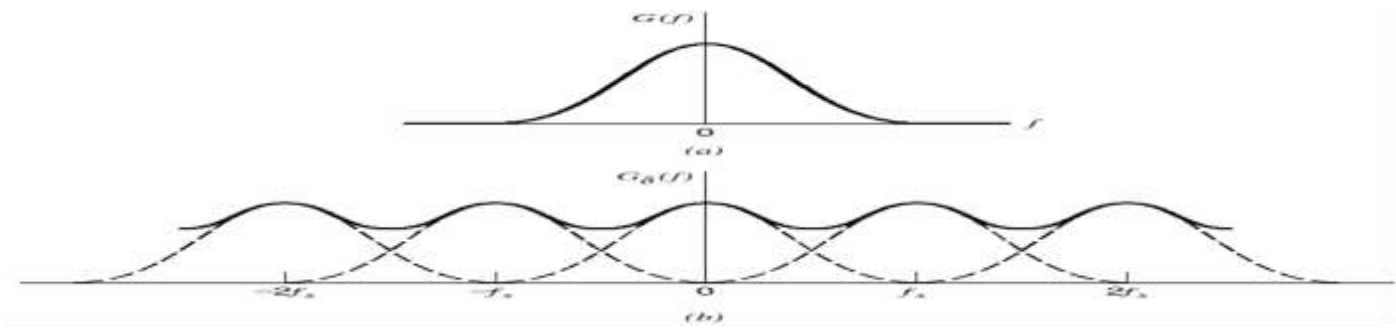


Figure 3.3 (*a*) Spectrum of a signal. (*b*) Spectrum of an undersampled version of the signal exhibiting the aliasing phenomenon.

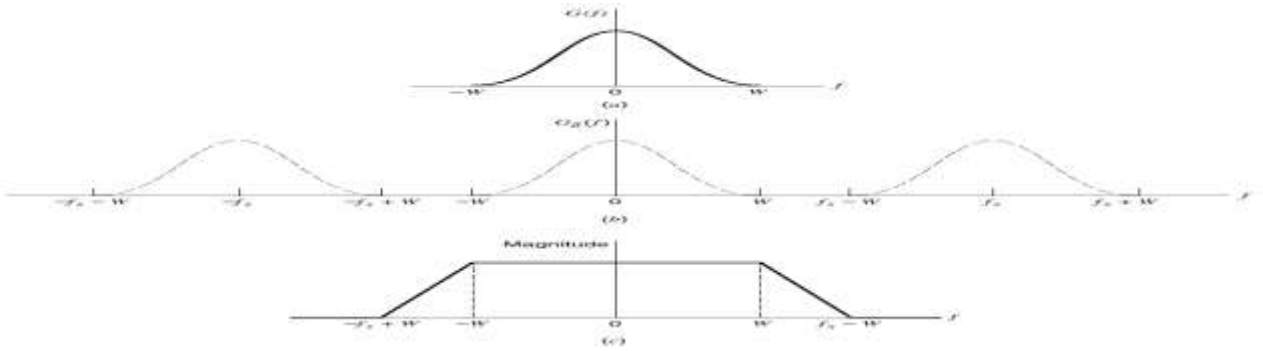
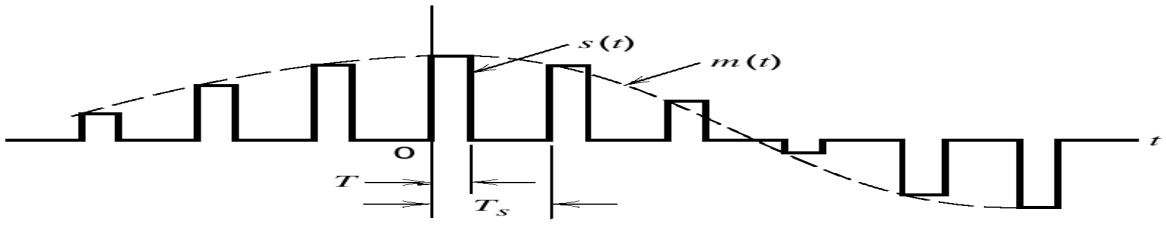


Figure 3.4 (*a*) Anti-alias filtered spectrum of an information-bearing signal. (*b*) Spectrum of instantaneously sampled version of the signal, assuming the use of a sampling rate greater than the Nyquist rate. (*c*) Magnitude response of reconstruction filter.

Pulse-Amplitude Modulation :



Let $s(t)$ denote the sequence of flat-top pulses as

$$s(t) = \sum_{n=-\infty}^{\infty} m(nT_s)\, h(t - nT_s) \qquad (3.10)$$

$$h(t) = \begin{cases} 1, & 0 < t < T \\ 1/2, & t = 0, t = T \\ 0, & \text{otherwise} \end{cases} \qquad (3.11)$$

The instantaneously sampled version of $m(t)$ is

$$m_\delta(t) = \sum_{n=-\infty}^{\infty} m(nT_s)\,\delta(t - nT_s) \qquad (3.12)$$

$$m_\delta(t) * h(t) = \int_{-\infty}^{\infty} m_\delta(\tau) h(t-\tau)\, d\tau$$

$$= \int_{-\infty}^{\infty} \sum_{n=-\infty}^{\infty} m(nT_s)\,\delta(\tau - nT_s) h(t-\tau)\, d\tau$$

$$= \sum_{n=-\infty}^{\infty} m(nT_s) \int_{-\infty}^{\infty} \delta(\tau - nT_s) h(t-\tau)\, d\tau \qquad (3.13)$$

Using the sifting property, we have

$$m_\delta(t) * h(t) = \sum_{n=-\infty}^{\infty} m(nT_s) h(t - nT_s) \qquad (3.14)$$

The PAM signal $s(t)$ is

$$s(t) = m_\delta(t) * h(t) \qquad (3.15)$$

$$\Leftrightarrow S(f) = M_\delta(f) H(f) \qquad (3.16)$$

Rec all (3.2) $g_\delta(t) \Leftrightarrow f_s \sum_{m=-\infty}^{\infty} G(f - mf_s)$ (3.2)

$$M_\delta(f) = f_s \sum_{k=-\infty}^{\infty} M(f - k f_s) \qquad (3.17)$$

$$S(f) = f_s \sum_{k=-\infty}^{\infty} M(f - k f_s) H(f) \qquad (3.18)$$

Pulse Amplitude Modulation – Natural and Flat-Top Sampling:

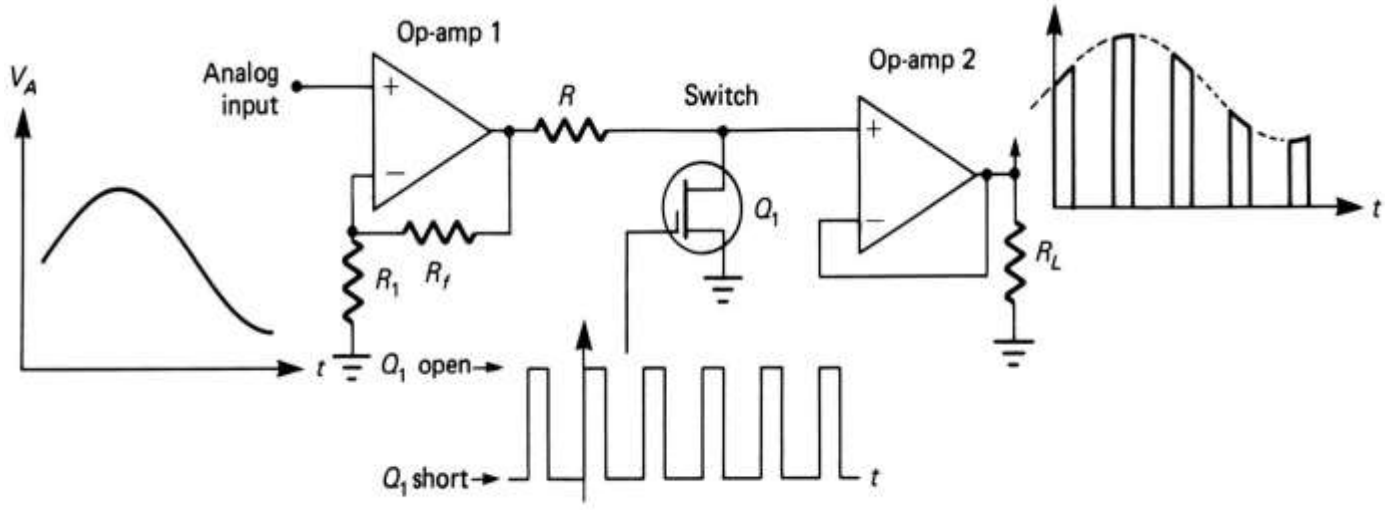
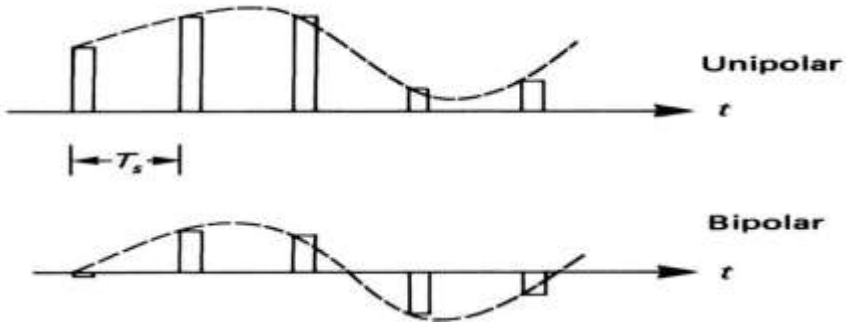


圖 11-3 脈衝幅度調變器，自然取樣。



- The most common technique for sampling voice in PCM systems is to a sample-and-hold circuit.

- The instantaneous amplitude of the analog (voice) signal is held as a constant charge on a capacitor for the duration of the sampling period *T*s.

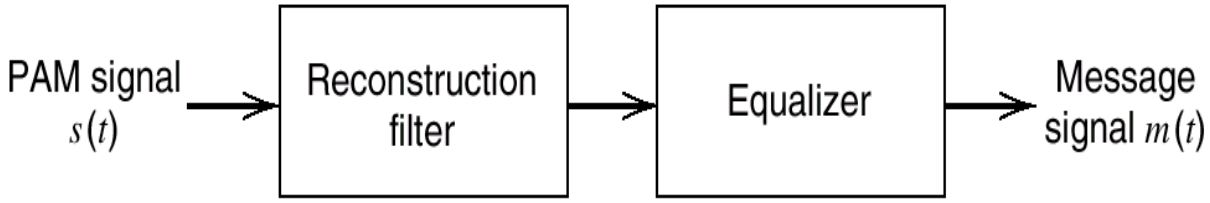
- This technique is useful for holding the sample constant while other processing is taking place, but it alters the frequency spectrum and introduces an error, called aperture

error, resulting in an inability to recover exactly the original analog signal.

- The amount of error depends on how mach the analog changes during the holding time, called aperture time.

- To estimate the maximum voltage error possible, determine the maximum slope of the analog signal and multiply it by the aperture time D*T*

Recovering the original message signal m(t) from PAM signal :



Where the filter bandwidth is W

The filter output is $f_s M(f)H(f)$. Note that the Fourier transform of $h(t)$ is given by

$$H(f) = T\,\mathrm{sinc}(f\,T)\exp(-j\pi f\,T) \qquad (3.19)$$

amplitude distortion delay $= T/2$

$\Rightarrow$ aparture effect

Let the equalizer response is

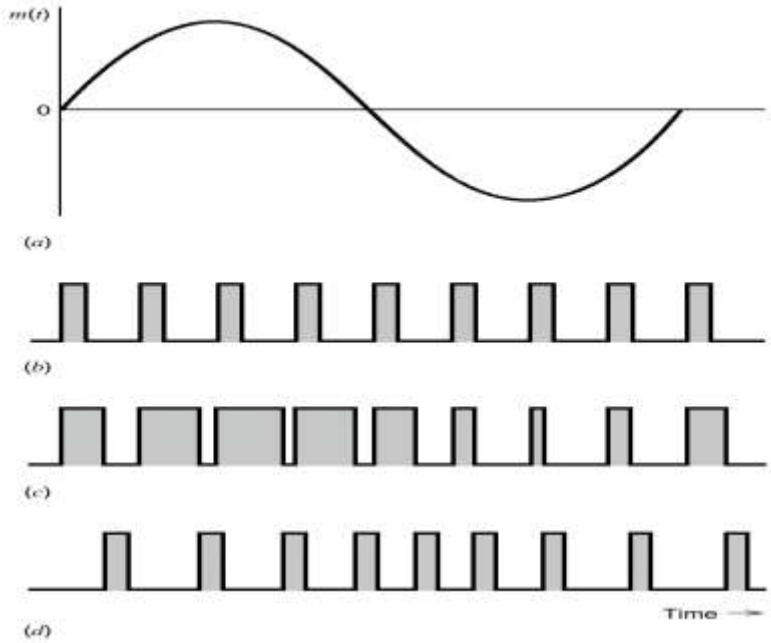
$$\frac{1}{H(f)} = \frac{1}{T\,\mathrm{sinc}(f\,T)} = \frac{\pi f}{\sin(\pi f\,T)} \qquad (3.20)$$

Ideally, the original signal $m(t)$ can be recovered completely

Other Forms of Pulse Modulation:

- In pulse width modulation (PWM), the width of each pulse is made directly proportional to the amplitude of the information signal.

- In pulse position modulation, constant-width pulses are used, and the position or time of occurrence of each pulse from some reference time is made directly proportional to the amplitude of the information signal.



Pulse Code Modulation (PCM) :

- Pulse code modulation (PCM) is produced by analog-to-digital conversion process.

- As in the case of other pulse modulation techniques, the rate at which samples are taken and encoded must conform to the Nyquist sampling rate.
- The sampling rate must be greater than, twice the highest frequency in the analog signal,

$$f_s > 2f_{A(max)}$$

Quantization Process:

(*a*) (*b*)

Define partition cell

$$\mathcal{J}_k : \{m_k < m \le m_{k+1}\}, \quad k = 1,2,\cdots, L \qquad (3.21)$$

Where m_k is the decision level or the decision threshold.

Amplitude quantization: The process of transforming the sample amplitude $m(nT_s)$ into a discrete amplitude $v(nT_s)$ as shown in Fig 3.9

If $m(t) \in \mathcal{J}_k$ then the quantizer output is v_k where v_k, $k = 1,2,\cdots, L$ are the representation or reconstruction levels, $m_{k+1} - m_k$ is the step size.

The mapping $v = g(m)$ (3.22)

is called the quantizer characteristic, which is a staircase function.

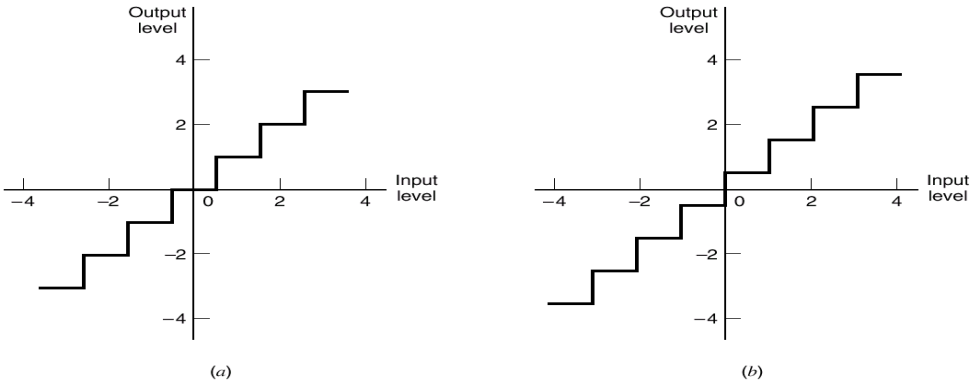


Figure 3.10 Two types of quantization: (*a*) midtread and (*b*) midrise.

Quantization Noise:

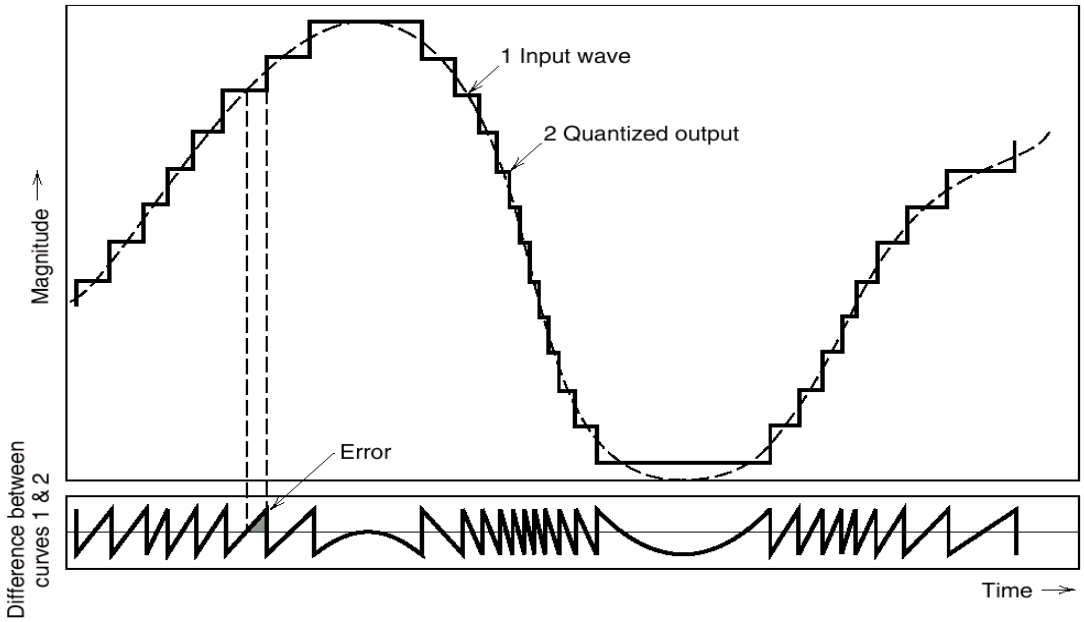


Figure 3.11 Illustration of the quantization process

Let the quantization error be denoted by the random variable Q of sample value q

$$q = m - \nu \qquad (3.23)$$

$$Q = M - V, \ (E[M] = 0) \qquad (3.24)$$

Assuming a uniform quantizer of the midrise type

the step-size is $\Delta = \frac{2m_{\max}}{L}$ (3.25)

$-m_{\max} < m < m_{\max}$, L: total number of levels

$$f_Q(q) = \begin{cases} \frac{1}{\Delta}, & -\frac{\Delta}{2} < q \le \frac{\Delta}{2} \\ 0, & \text{otherwise} \end{cases} \qquad (3.26)$$

$$\sigma_Q^2 = E[Q^2] = \int_{-\frac{\Delta}{2}}^{\frac{\Delta}{2}} q^2 f_Q(q) dq = \frac{1}{\Delta} \int_{-\frac{\Delta}{2}}^{\frac{\Delta}{2}} q^2 dq$$

$$= \frac{\Delta^2}{12} \qquad (3.28)$$

When the quatized sample is expressed in binary form,

$$L = 2^R \qquad (3.29)$$

where R is the number of bits per sample

$$R = \log_2 L \qquad (3.30)$$

$$\Delta = \frac{2m_{\max}}{2^R} \qquad (3.31)$$

$$\sigma_Q^2 = \frac{1}{3} m_{\max}^2 2^{-2R} \qquad (3.32)$$

Let P denote the average power of $m(t)$

$$\Rightarrow (SNR)_o = \frac{P}{\sigma_Q^2}$$

$$= \left(\frac{3P}{m_{\max}^2}\right) 2^{2R} \qquad (3.33)$$

▶ EXAMPLE 3.1 Sinusoidal Modulating Signal

Consider the special case of a full-load sinusoidal modulating signal of amplitude A_m, which utilizes all the representation levels provided. The average signal power is (assuming a load of 1 ohm)

$$P = \frac{A_m^2}{2}$$

The total range of the quantizer input is $2A_m$, because the modulating signal swings between $-A_m$ and A_m. We may therefore set $m_{max} = A_m$, in which case the use of Equation (3.32) yields the average power (variance) of the quantization noise as

$$\sigma_Q^2 = \tfrac{1}{3}A_m^2 2^{-2R}$$

Thus the output signal-to-noise ratio of a uniform quantizer, for a full-load test tone, is

$$(\text{SNR})_O = \frac{A_m^2/2}{A_m^2 2^{-2R}/3} = \frac{3}{2}(2^{2R}) \tag{3.34}$$

Expressing the signal-to-noise ratio in decibels, we get

$$10 \log_{10}(\text{SNR})_O = 1.8 + 6R \tag{3.35}$$

Pulse Code Modulation (PCM):

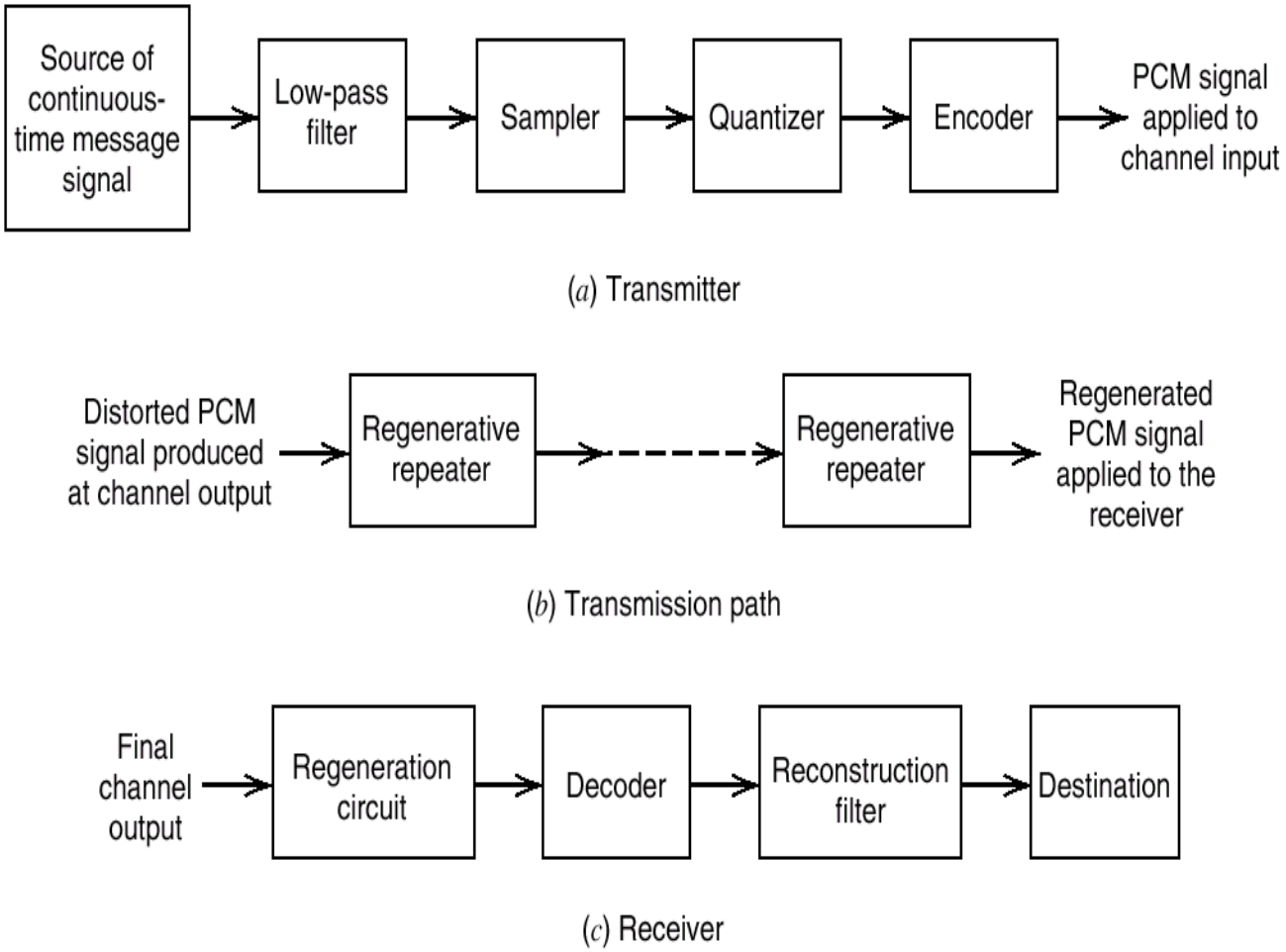
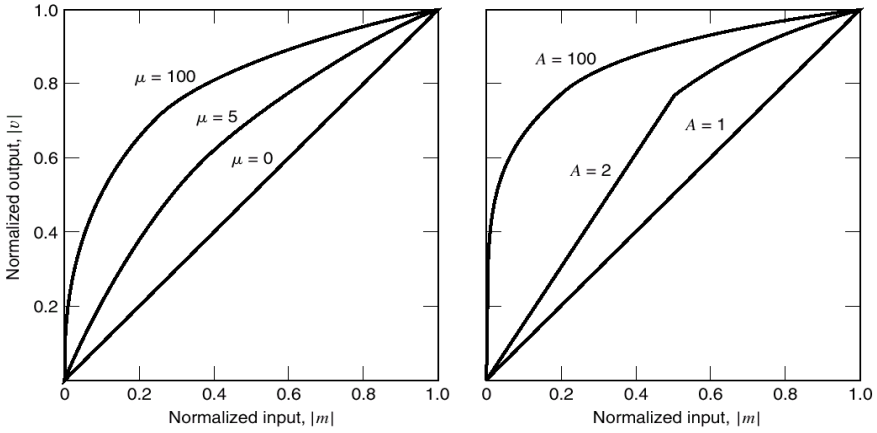


Figure 3.13 The basic elements of a PCM system

Quantization (nonuniform quantizer):



Compression laws. (*a*) *m* -law. (*b*) A-law.

μ - law

$$|v| = \frac{\log(1+\mu|m|)}{\log(1+\mu)} \qquad (3.48)$$

$$\frac{d|m|}{d|v|} = \frac{\log(1+\mu)}{\mu}(1+\mu|m|) \qquad (3.49)$$

A - law

$$|v| = \begin{cases} \dfrac{A(m)}{1+\log A} & 0 \le |m| \le \dfrac{1}{A} \\ \dfrac{1+\log(A|m|)}{1+\log A} & \dfrac{1}{A} \le |m| \le 1 \end{cases} \qquad (3.50)$$

$$\frac{d|m|}{d|v|} = \begin{cases} \dfrac{1+\log A}{A} & 0 \le |m| \le \dfrac{1}{A} \\ (1+A)|m| & \dfrac{1}{A} \le |m| \le 1 \end{cases} \qquad (3.51)$$

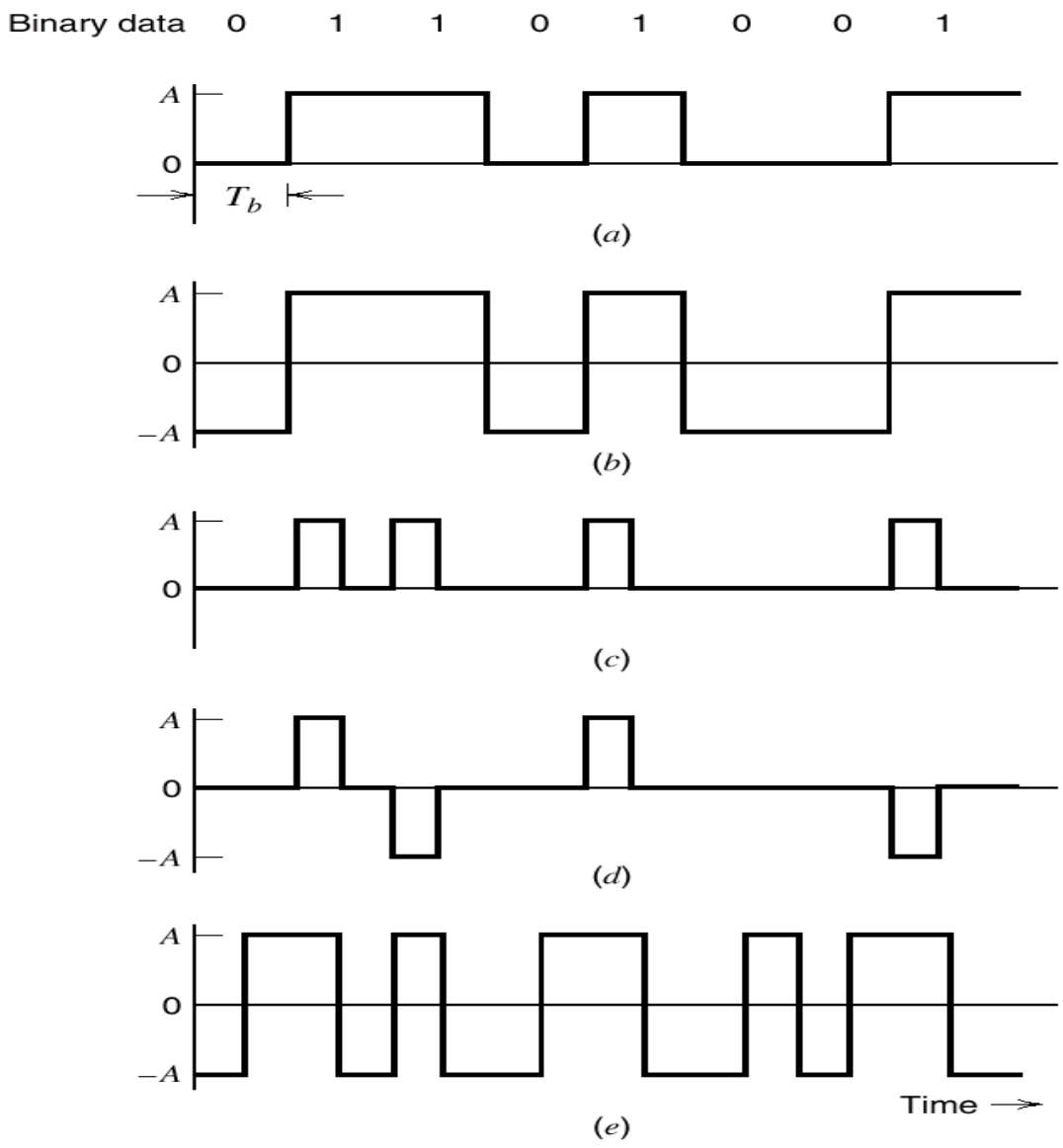


Figure 3.15 Line codes for the electrical representations of binary data.
(*a*) Unipolar NRZ signaling. (*b*) Polar NRZ signaling.
(c) Unipolar RZ signaling. (*d*) Bipolar RZ signaling.
(*e*) Split-phase or Manchester code.

Noise consideration in PCM systems:

(Channel noise, quantization noise)

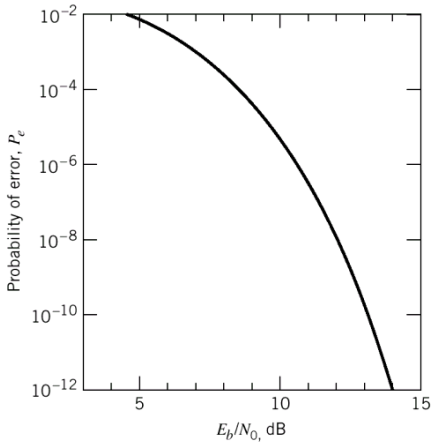
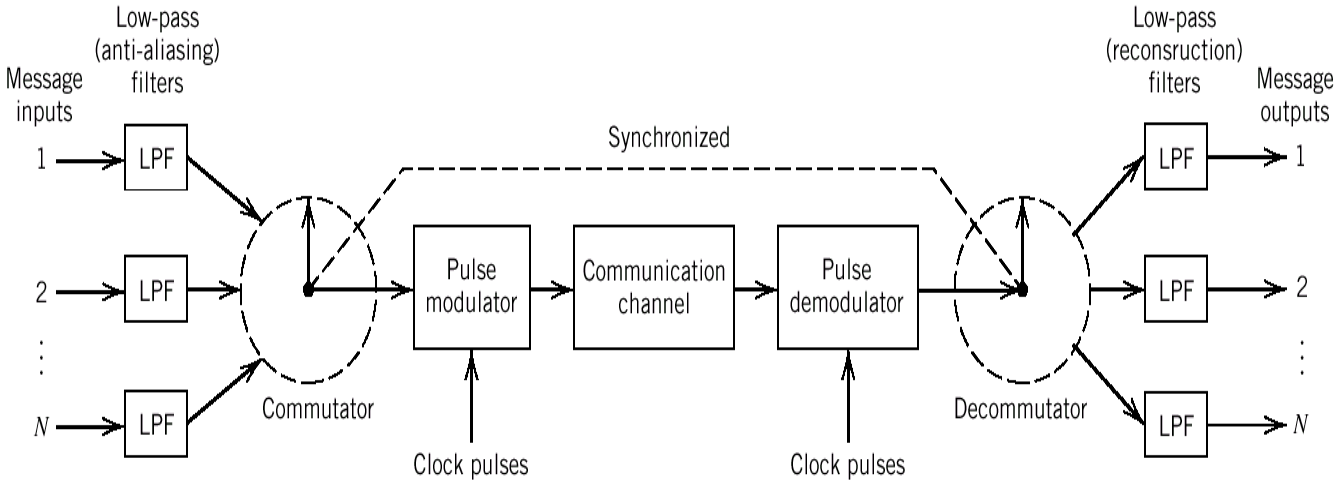


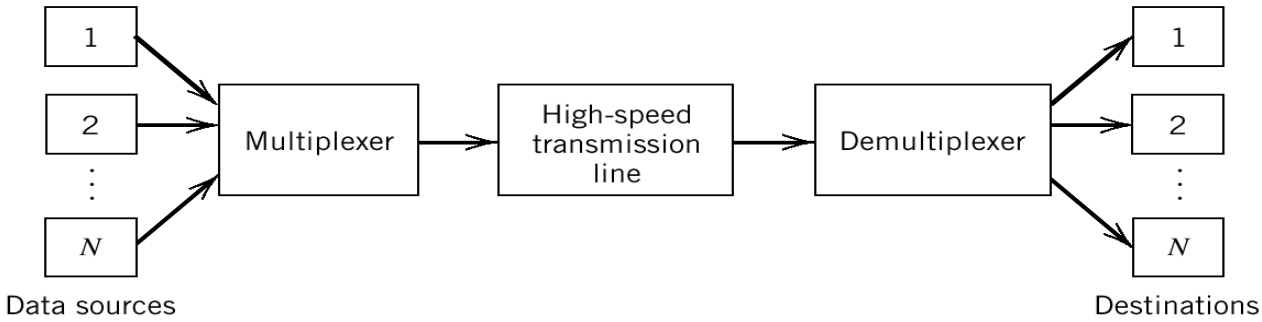
TABLE 3.3 *Influence of E_b/N_0 on the probability of error*

E_b/N_0	*Probability of Error* P_e	*For a Bit Rate of 10^5 b/s, This Is About One Error Every*
4.3 dB	10^{-2}	10^{-3} second
8.4	10^{-4}	10^{-1} second
10.6	10^{-6}	10 seconds
12.0	10^{-8}	20 minutes
13.0	10^{-10}	1 day
14.0	10^{-12}	3 months

Time-Division Multiplexing(TDM):



Digital Multiplexers :

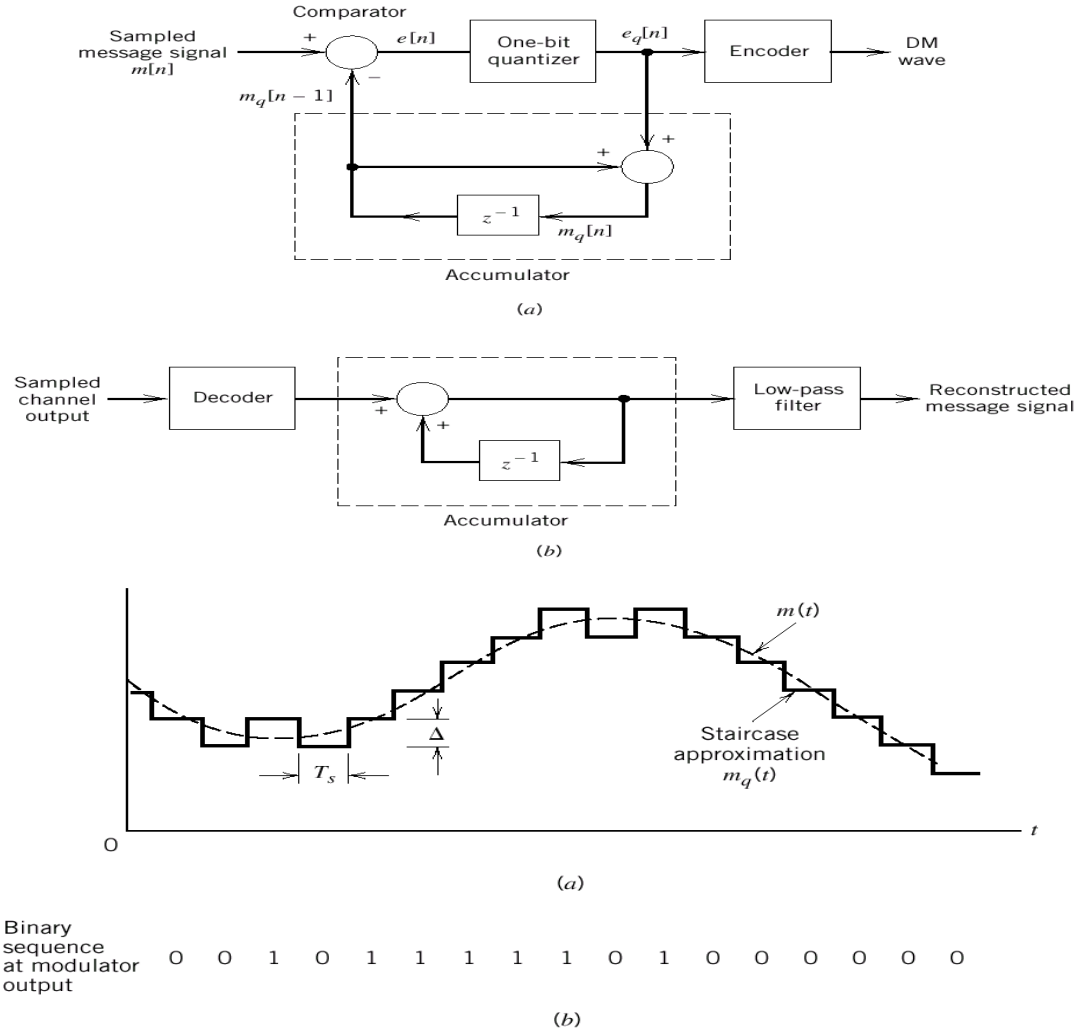


Virtues, Limitations and Modifications of PCM:

Advantages of PCM

1. Robustness to noise and interference
2. Efficient regeneration
3. Efficient SNR and bandwidth trade-off
4. Uniform format
5. Ease add and drop
6. Secure

Delta Modulation (DM) :



Let $m[n] = m(nT_s)$, $n = 0, \pm 1, \pm 2, \ldots$

where T_s is the sampling period and $m(nT_s)$ is a sample of $m(t)$.

The error signal is $e[n]$

$$= m[n] - m_q[n-1] \quad e_q \qquad (3.52)$$

$$[n] = \Delta \operatorname{sgn}(e[n]) \qquad (3.53)$$

$$m_q[n] = m_q[n-1] + e_q[n] \qquad (3.54)$$

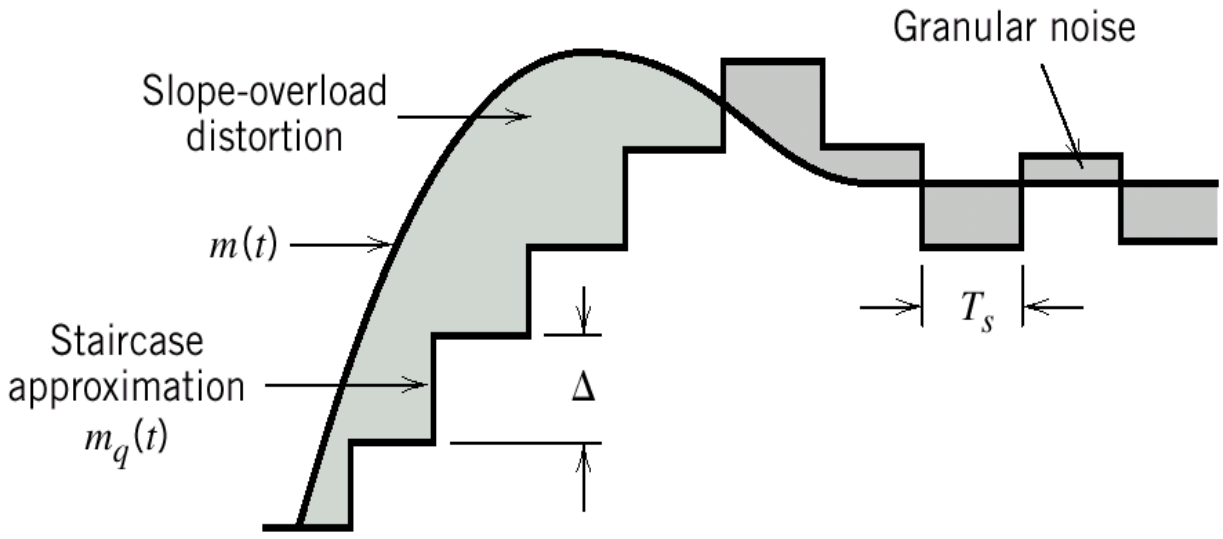
where $m_q[n]$ is the quantizer output, $e_q[n]$ is the quantized version of $e[n]$, and Δ is the step size

The modulator consists of a comparator, a quantizer, and an accumulator

The output of the accumulator is

$$m_q[n] = \Delta \sum_{i=1}^{n} \text{sgn}(e[i])$$

$$= \sum_{i=1}^{n} e_q[i] \qquad (3.55)$$



Two types of quantization errors :

Slope overload distortion and granular noise

Slope Overload Distortion and Granular Noise:

Denote the quantization error by $q[n]$,

$$m_q[n] = m[n] - q[n] \qquad (3.56)$$

Recall (3.52), we have

$$e[n] = m[n] - m[n-1] - q[n-1] \qquad (3.57)$$

Except for $q[n-1]$, the quantizer input is a first backward difference of the input signal

To avoid slope-overload distortion, we require

$$\text{(slope)} \qquad \frac{\Delta}{T_s} \geq \max\left|\frac{dm(t)}{dt}\right| \qquad (3.58)$$

On the other hand, granular noise occurs when step size Δ is too large relative to the local slope of $m(t)$.

Delta-Sigma modulation (sigma-delta modulation):

The $\Delta - \Sigma$ modulation which has an **integrator** can relieve the draw back of delta modulation (**differentiator**)

Beneficial effects of using integrator:

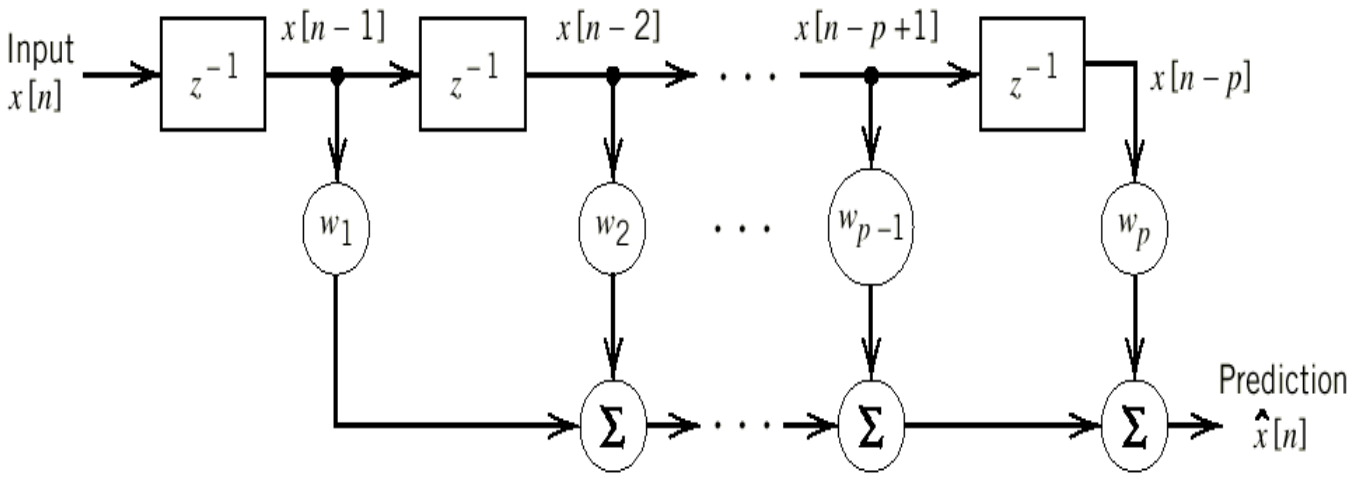
1. Pre-emphasize the low-frequency content
2. Increase correlation between adjacent samples (reduce the variance of the error signal at the quantizer input)
3. Simplify receiver design

Because the transmitter has an integrator, the receiver consists simply of a low-pass filter.

(The differentiator in the conventional DM receiver is cancelled by the integrator)

Linear Prediction (to reduce the sampling rate):

Consider a finite-duration impulse response (FIR) discrete-time filter which consists of three blocks :
1. Set of *p* (***p*: prediction order**) unit-delay elements (*z*-1)
2. Set of multipliers with coefficients *w*1,*w*2,…*wp*
3. Set of adders ($\sum$)



The filter output (The linear predition of the input) is

$$\hat{x}[n] = \sum_{k=1}^{p} w_k \, x(n-k) \qquad (3.59)$$

The predic tion error is

$$e[n] = x[n] - \hat{x}[n] \qquad (3.60)$$

Let the index of performanœ be

$$J = E\left[e^2[n]\right] \text{ (mean squareerror)} \quad (3.61)$$

Find $w_1, w_2, \ldots, w_p$ to minimize J

From (3.59)(3.60)and (3.61) wehave

$$J = E\left[x^2[n]\right] - 2\sum_{k=1}^{p} w_k E[x[n]x[n-k]]$$

$$+ \sum_{j=1}^{p}\sum_{k=1}^{p} w_j w_k E[x[n-j]x[n-k]] \qquad (3.62)$$

Assume $X(t)$ is stationary process with zero mean ($E[x[n]] = 0$)

$$\sigma_X^2 = E\left[x^2[n]\right] - (E[x[n]])^2$$
$$= E\left[x^2[n]\right]$$

The autocorrelation

$$R_X(\tau = kT_s) = R_X[k] = E\left[x[n]x[n-k]\right]$$

We may simplify J as

$$J = \sigma_X^2 - 2\sum_{k=1}^{p} w_k R_X[k] + \sum_{j=1}^{p}\sum_{k=1}^{p} w_j w_k R_X[k-j] \qquad (3.63)$$

$$\frac{\partial J}{\partial w_k} = -2R_X[k] + 2\sum_{j=1}^{p} w_j R_X[k-j] = 0$$

$$\sum_{j=1}^{p} w_j R_X[k-j] = R_X[k] = R_X[-k], \quad k = 1,2,\ldots,p \qquad (3.64)$$

(3.64) are called Wiener-Hopf equations

as, if $\mathbf{R}_X^{-1}$ exists $\quad \mathbf{w}_0 = \mathbf{R}_X^{-1}\mathbf{r}_X \qquad (3.66)$

where $\quad \mathbf{w}_0 = \left[w_1, w_2, \ldots, w_p\right]^T$

$$\mathbf{r}_X = [R_X[1], R_X[2], \ldots, R_X[p]]^T$$

$$\mathbf{R}_X = \begin{bmatrix} R_X[0] & R_X[1] & \cdots & R_X[p-1] \\ R_X[1] & R_X[0] & \cdots & R_X[p-2] \\ \vdots & \vdots & & \vdots \\ R_X[p-1] & R_X[p-2] & \cdots & R_X[0] \end{bmatrix}$$

$$R_X[0], R_X[1], \cdots, R_X[p]$$

Substituting (3.64) into (3.63) yields

$$J_{\min} = \sigma_X^2 - 2\sum_{k=1}^{p} w_k R_X[k] + \sum_{k=1}^{p} w_k R_X[k]$$

$$= \sigma_X^2 - \sum_{k=1}^{p} w_k R_X[k]$$

$$= \sigma_X^2 - \mathbf{r}_X^T\mathbf{w}_0 = \sigma_X^2 - \mathbf{r}_X^T\mathbf{R}_X^{-1}\mathbf{r}_X \qquad (3.67)$$

$\because \mathbf{r}^T\mathbf{R}^{-1}\mathbf{r} \geq 0, \therefore J$ is always less than σ^2

$X \quad X \quad X \qquad \min \qquad X$

Linear adaptive prediction :

The predictor is adaptive in the follow sense

1. Compute w_k, $k = 1,2,\ldots,p$, starting any initial values
2. Do iteration using the method of steepest descent

Define the gradient vector

$$g_k = \frac{\partial J}{\partial w_k}, \quad k = 1,2,\ldots,p \qquad (3.68)$$

$w_k[n]$ denotes the value at iteration n . Then update $w_k[n+1]$

$$w_k[n+1] = w_k[n] - \frac{1}{2}\mu g_k, \quad k = 1,2,\ldots,p \qquad (3.69)$$

where μ is a step-size parameter and $\frac{1}{2}$ is for convenience of presentation.

$$g_k = \frac{\partial J}{\partial w_k} = -2R_X[k] + 2\sum_{j=1}^{P} w_j R_X[k-j]$$

$$= -2E[x[n]x[n-k]] + 2\sum_{j=1}^{p} w_j E[x[n-j]x[n-k]], \quad k = 1,2,\ldots,p \qquad (3.70)$$

To simplify the computing we use $x[n]x[n-k]$ for E[x[n]x[n-k]] (ignore the expectation)

$$\hat{g}_k[n] = -2x[n]x[n-k] + 2\sum_{j=1}^{p} w_j[n]x[n-j]x[n-k], \quad k = 1,2,\ldots,p \qquad (3.71)$$

$$\hat{w}_k[n+1] = \hat{w}_k[n] + \mu x[n-k]\left(x[n] - \sum_{j=1}^{p} \hat{w}_j[n]x[n-j]\right)$$

$$= \hat{w}_k[n] + \mu x[n-k]e[n], \quad k = 1,2,\ldots,p \qquad (3.72)$$

where $e[n] = x[n] - \sum_{j=1}^{p} \hat{w}_j[n]x[n-j]$ by (3.59)+(3.60) (3.73)

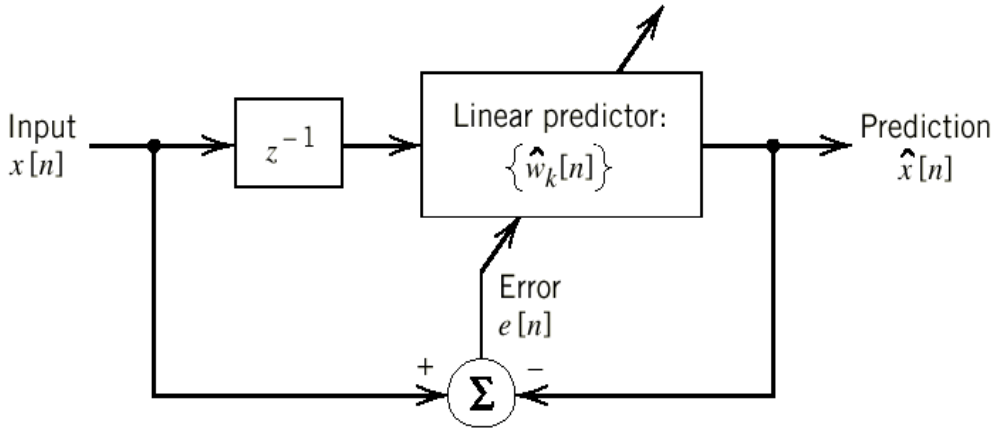
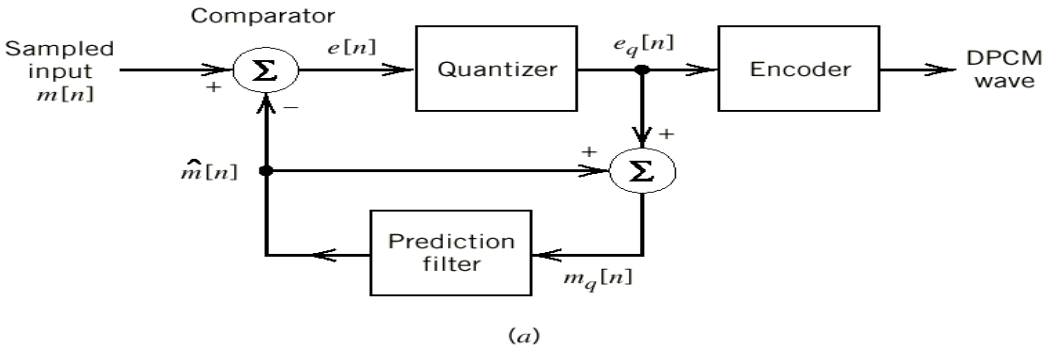


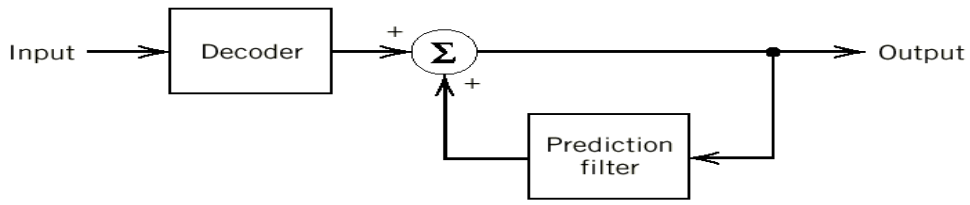
Figure 3.27
Block diagram illustrating the linear adaptive prediction process

Differential Pulse-Code Modulation (DPCM):

Usually **PCM** has the sampling rate higher than the **Nyquist rate** .The encode signal contains redundant information. **DPCM** can efficiently remove this redundancy.



(a)



(b)

Figure 3.28 DPCM system. (*a*) Transmitter. (*b*) Receiver.

Input signal to the quantizer is defined by:

$$e[n] = m[n] - \hat{m}[n] \qquad (3.74)$$

$\hat{m}[n]$ is a prediction value.

The quantizer output is

$$e_q[n] = e[n] + q[n] \qquad (3.75)$$

where $q[n]$ is quantization error.

The prediction filter input is

$$m_q[n] = \hat{m}[n] + e[n] + q[n] \qquad (3.77)$$

From (3.74) $\searrow$

$m[n]$

$$\Rightarrow m_q[n] = m[n] + q[n] \qquad (3.78)$$

Processing Gain:

The $(SNR)_o$ of the DPCM system is

$$(SNR)_o = \frac{\sigma_M^2}{\sigma_Q^2} \qquad (3.79)$$

where σ_M^2 and σ_Q^2 are variances of $m[n]$ $(E[m[n]] = 0)$ and $q[n]$

$$(SNR)_o = \left(\frac{\sigma_M^2}{\sigma_E^2}\right)\left(\frac{\sigma_E^2}{\sigma_Q^2}\right)$$

$$= G_p (SNR)_Q \qquad (3.80)$$

where σ_E^2 is the variance of the predictions error

and the signal - to - quantization noise ratio is

$$(SNR)_Q = \frac{\sigma_E^2}{\sigma_Q^2} \qquad (3.81)$$

Processing Gain, $G_p = \frac{\sigma_M^2}{\sigma_E^2}$ (3.82)

Design a prediction filter to maximize G_p (minimize σ_E^2)

Adaptive Differential Pulse-Code Modulation (ADPCM):

Need for coding speech at low bit rates , we have two aims in mind:

1. Remove redundancies from the speech signal as far as possible.

2. Assign the available bits in a perceptually efficient manner.

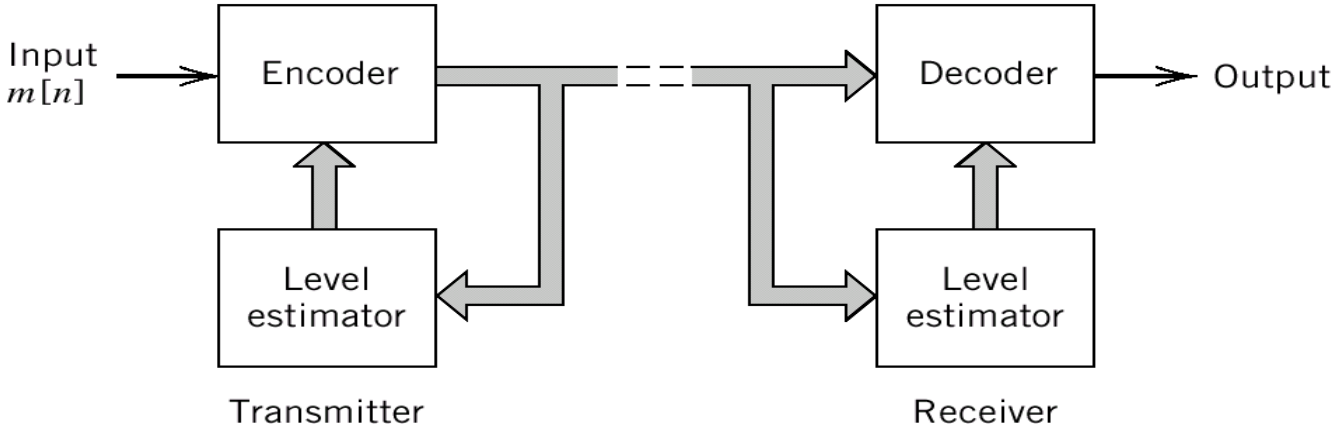


Figure 3.29 Adaptive quantization with backward estimation (AQB).

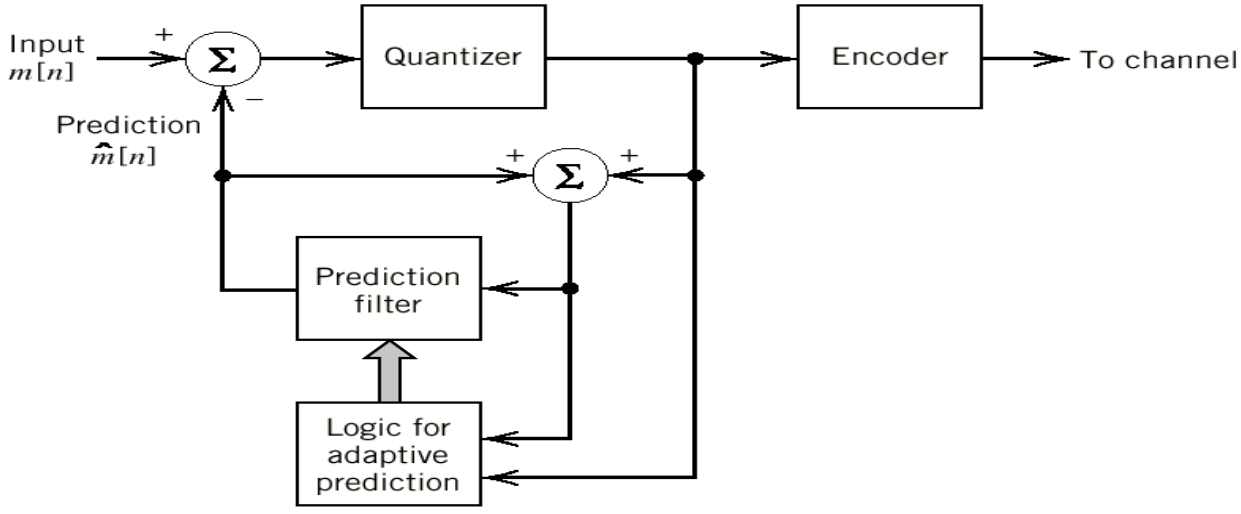


Figure 3.30 Adaptive prediction with backward estimation (APB).

UNIT III BASEBAND TRANSMISSION

CORRELATIVE LEVEL CODING:

- Correlative-level coding (partial response signaling)
  - adding ISI to the transmitted signal in a controlled manner
- Since ISI introduced into the transmitted signal is known, its effect can be interpreted at the receiver
- A practical method of achieving the theoretical maximum signaling rate of 2W symbol per second in a bandwidth of W Hertz
- Using realizable and perturbation-tolerant filters

Duo-binary Signaling :

Duo : doubling of the transmission capacity of a straight binary system

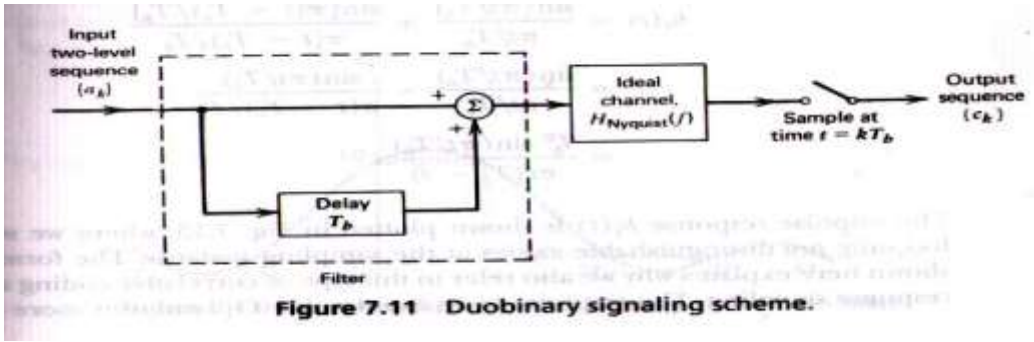
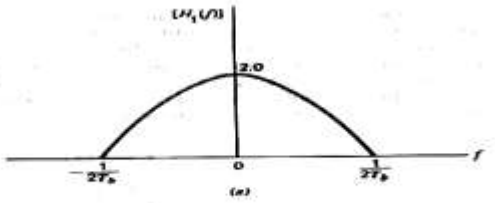


Figure 7.11 Duobinary signaling scheme.

- Binary input sequence {bk} : uncorrelated binary symbol 1, 0

$$a_k = \begin{cases} +1 & \text{if symbol } b_k \text{ is } 1 \\ -1 & \text{if symbol } b_k \text{ is } 0 \end{cases}$$

$$c_k = a_k + a_{k-1}$$



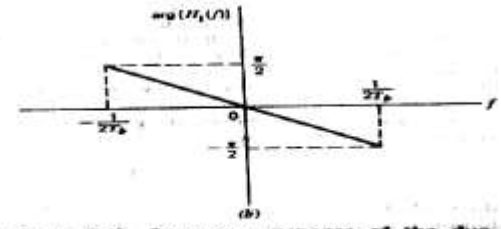


Figure 7.12 Frequency response of the duobinary conversion filter. (a) Amplitude response. (b) Phase response.

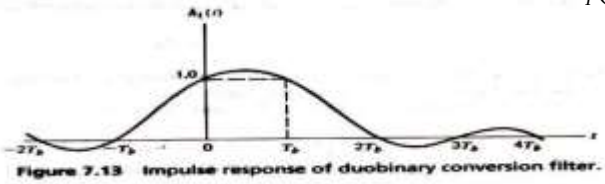


Figure 7.13 Impulse response of duobinary conversion filter.

$$H_I(f) = H_{Nyquist}(f)[1+\exp(-j2\pi fT_b)]$$
$$= H_{Nyquist}(f)[\exp(j\pi fT_b)+\exp(-j\pi fT_b)]\exp(-j\pi fT_b)$$
$$= 2H_{Nyquist}(f)\cos(\pi fT_b)\exp(-j\pi fT_b)$$

$$H_{Nyquist}(f) = \begin{cases} 1, & |f| \le 1/2T_b \\ 0, & otherwise \end{cases}$$

$$H_I(f) = \begin{cases} 2\cos(\pi fT_b)\exp(-j\pi fT_b), & |f| \le 1/2T_b \\ 0, & otherwise \end{cases}$$

$$h_I(t) = \frac{\sin(\pi t/T_b)}{\pi t/T_b} + \frac{\sin[\pi(t-T_b)/T_b]}{\pi(t-T_b)/T_b}$$
$$= \frac{T_b^2 \sin(\pi t/T_b)}{\pi t(T_b - t)}$$

- The tails of hI(t) decay as 1/|t|2, which is a faster rate of decay than 1/|t| encountered in the ideal Nyquist channel.
- Let represent the estimate of the original pulse ak as conceived by the receiver at time t=kTb
- Decision feedback : technique of using a stored estimate of the previous symbol
- Propagate : drawback, once error are made, they tend to propagate through the output
- Precoding : practical means of avoiding the error propagation phenomenon before the duobinary coding

$$d_k = b_k \oplus d_{k-1}$$

$$d_k = \begin{cases} symbol\ 1 & if\ either\ symbol\ \ b_k \ \ or\ d_{k-1}\ is\ \ 1 \\ symbol\ 0 & otherwise \end{cases}$$

- {dk} is applied to a pulse-amplitude modulator, producing a corresponding two-level sequence of short pulse {ak}, where +1 or –1 as before

$$c_k = a_k + a_{k-1}$$

$$c_k = \begin{cases} 0 & if\ data\ symbol\ \ b_k\ is\ 1 \\ \pm 2 & if\ data\ symbol\ \ b_k\ is\ 0 \end{cases}$$

- |ck|=1 : random guess in favor of symbol 1 or 0

- |ck|=1 : random guess in favor of symbol 1 or 0

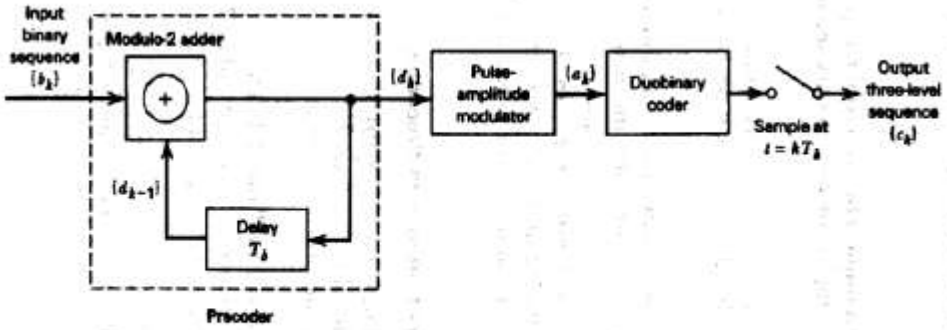


Figure 7.14 A precoded duobinary scheme; details of the duobinary coder are given in Figure 7.11.

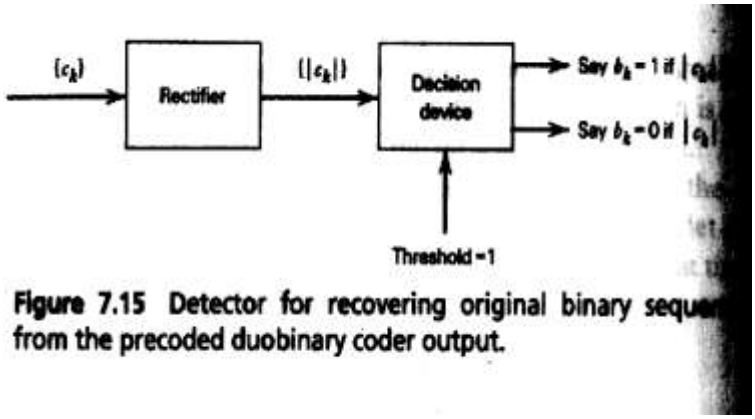


Figure 7.15 Detector for recovering original binary sequ from the precoded duobinary coder output.

Modified Duo-binary Signaling :

- Nonzero at the origin : undesirable
- Subtracting amplitude-modulated pulses spaced 2Tb second

$$c_k = a_k + a_{k-1}$$

$$H_{IV}(f) = H_{Nyquist}(f)[1-\exp(-j4\pi fT_b)]$$
$$= 2jH_{Nyquist}(f)\sin(2\pi fT_b)\exp(-j2\pi fT_b)$$

$$H_{IV}(f) = \begin{cases} 2j\sin(2\pi fT_b)\exp(-j2\pi fT_b), & |f| \le 1/2T_b \\ 0, & elsewhere \end{cases}$$

$$h_{IV}(t) = \frac{\sin(\pi t/T_b)}{\pi t/T_b} - \frac{\sin[\pi(t-2T_b)/T_b]}{\pi(t-2T_b)/T_b}$$
$$= \frac{2T_b^2\sin(\pi t/T_b)}{\pi t(2T_b - t)}$$

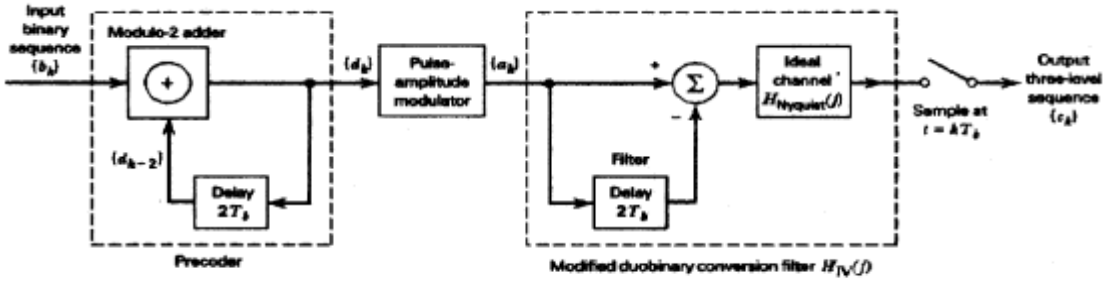


Figure 7.16 Modified duobinary signaling scheme.

- **precoding**

$$d_k = b_k \oplus d_{k-2}$$
$$= \begin{cases} symbol\ 1 & if\ either\ symbol\ \ b_k\ \ or\ d_{k-2}\ is\ \ 1 \\ symbol\ 0 & otherwise \end{cases}$$

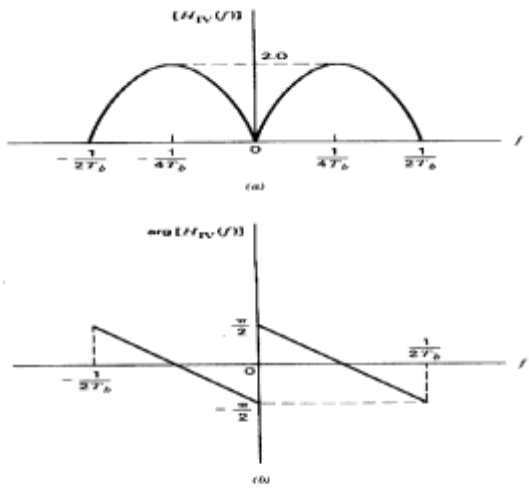


Figure 7.17 Frequency response of modified duobinary conversion filter. (a) Amplitude response. (b) Phase response.

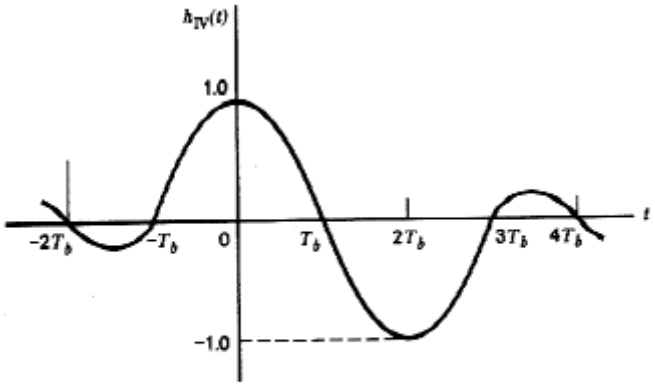


Figure 7.18 Impulse response of the modified duobinary conversion filter.

- |ck|=1 : random guess in favor of symbol 1 or 0

$$If \ |c_k|>1, \ say \ symbol \ b_k \ is \ 1$$

$$If \ |c_k|<1, \ say \ symbol \ b_k \ is \ 0$$

Generalized form of correlative-level coding:

- |ck|=1 : random guess in favor of symbol 1 or 0

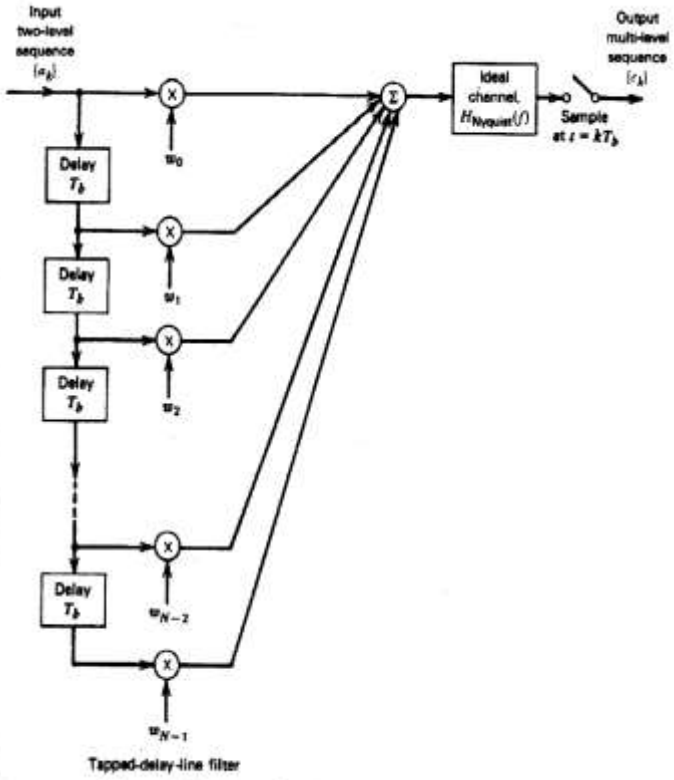
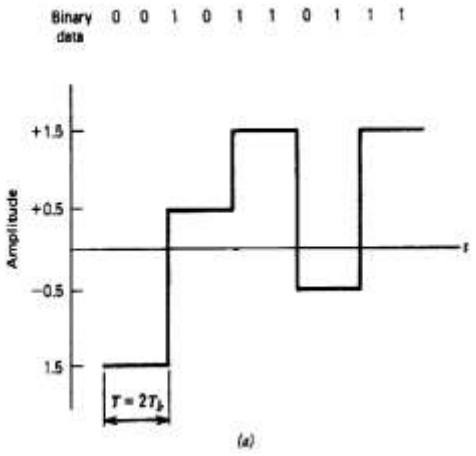


Figure 7.19 Generalized correlative coding scheme.

$$h(t) = \sum_{n}^{N-1} w_n \sin c\left(\frac{t}{T_b} - n\right)$$

Baseband M-ary PAM Transmission:



Dibit	Amplitude
00	−1.5
01	−0.5
10	+0.5
11	+1.5

(b)

Figure 7.20 Output of a quaternary system. (*a*) Waveform. (*b*) Representation of the 4 possible dibits.

- Produce one of M possible amplitude level
- T : symbol duration
- 1/T: signaling rate, symbol per second, bauds
  - Equal to log2M bit per second
- Tb : bit duration of equivalent binary PAM :
- To realize the same average probability of symbol error, transmitted power must be increased by a factor of M2/log2M compared to binary PAM

Tapped-delay-line equalization :

- Approach to high speed transmission
  - Combination of two basic signal-processing operation
  - Discrete PAM
  - Linear modulation scheme
- The number of detectable amplitude levels is often limited by ISI
- Residual distortion for ISI : limiting factor on data rate of the system

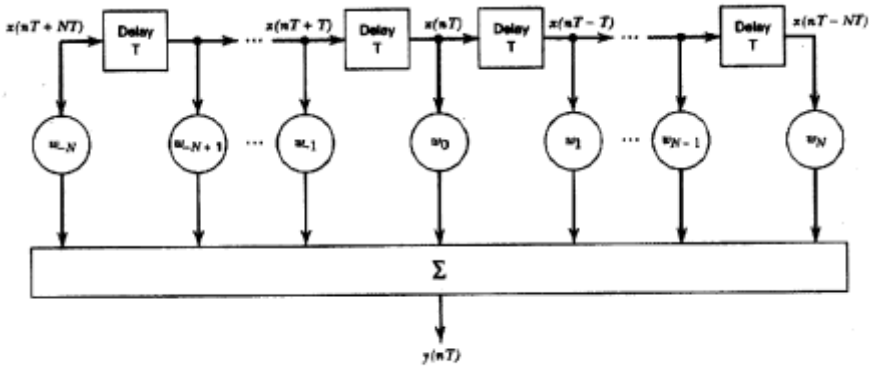


Figure 7.22 Tapped-delay-line filter.

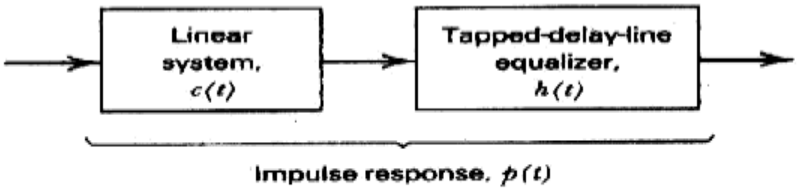


Figure 7.23 Cascade connection of linear system and tapped-delay-line equalizer.

- Equalization : to compensate for the residual distortion
- Equalizer : filter
  - A device well-suited for the design of a linear equalizer is the tapped-delay-line filter
  - Total number of taps is chosen to be (2N+1)

$$h(t) = \sum_{k=-N}^{N} w_k \delta(t - kT)$$

- P(t) is equal to the convolution of c(t) and h(t)

$$p(t) = c(t) * h(t) = c(t) * \sum_{k=-N}^{N} w_k \delta(t - kT)$$

$$= \sum_{k=-N}^{N} w_k c(t) * \delta(t - kT) = \sum_{k=-N}^{N} w_k c(t - kT)$$

- nT=t sampling time, discrete convolution sum

$$p(nT) = \sum_{k=-N}^{N} w_k c((n-k)T)$$

- Nyquist criterion for distortionless transmission, with T used in place of Tb, normalized condition p(0)=1

$$p(nT) = \begin{cases} 1, & n = 0 \\ 0, & n \neq 0 \end{cases} = \begin{cases} 1, & n = 0 \\ 0, & n = \pm 1, \pm 2, ..., \pm N \end{cases}$$

- Zero-forcing equalizer
  - Optimum in the sense that it minimizes the peak distortion(ISI) – worst case
  - Simple implementation
  - The longer equalizer, the more the ideal condition for distortionless transmission

Adaptive Equalizer :

- The channel is usually time varying
  - Difference in the transmission characteristics of the individual links that may be switched together
  - Differences in the number of links in a connection

- Adaptive equalization
  - Adjust itself by operating on the the input signal
- Training sequence
  - Precall equalization
  - Channel changes little during an average data call
- Prechannel equalization
  - Require the feedback channel
- Postchannel equalization
- synchronous
  - Tap spacing is the same as the symbol duration of transmitted signal

Least-Mean-Square Algorithm:

- Adaptation may be achieved
  - By observing the error b/w desired pulse shape and actual pulse shape
  - Using this error to estimate the direction in which the tap-weight should be changed
- Mean-square error criterion
  - More general in application
  - Less sensitive to timing perturbations
- : desired response, : error signal, : actual response
- Mean-square error is defined by cost fuction

$$\varepsilon = E \quad e_n^2$$

- Ensemble-averaged cross-correlation

$$\frac{\partial \varepsilon}{\partial w_k} = 2E\left[e_n \frac{\partial e_n}{\partial w_k} \right] = -2E\left[e_n \frac{\partial y_n}{\partial w_k} \right] = -2E\left[e_n x_{n-k} \right] = -2R_{ex}(k)$$

$$R_{ex}(k) = E\left[e_n x_{n-k} \right]$$

- Optimality condition for minimum mean-square error

$$\frac{\partial \varepsilon}{\partial w_k} = 0 \quad for\, k = 0, \pm 1, ..., \pm N$$

- Mean-square error is a second-order and a parabolic function of tap weights as a multidimentional bowl-shaped surface
- Adaptive process is a successive adjustments of tap-weight seeking the bottom of the bowl(minimum value)
- Steepest descent algorithm
  - The successive adjustments to the tap-weight in direction opposite to the vector of gradient)
  - Recursive formular (μ : step size parameter)

$$\begin{aligned} w_k(n+1) &= w_k(n) - \frac{1}{2}\mu \frac{\partial \varepsilon}{\partial w_k}, \quad k = 0, \pm 1, ..., \pm N \\ &= w_k(n) - \mu R_{ex}(k), \quad k = 0, \pm 1, ..., \pm N \end{aligned}$$

- Least-Mean-Square Algorithm
  - Steepest-descent algorithm is not available in an unknown environment
  - Approximation to the steepest descent algorithm using instantaneous estimate

$$\hat{R}_{ex}(k) = e_n x_{n-k}$$
$$\hat{w}_k(n+1) = \hat{w}_k(n) + \mu e_n x_{n-k}$$

- LMS is a feedback system

- In the case of small μ, roughly similar to steepest descent algorithm

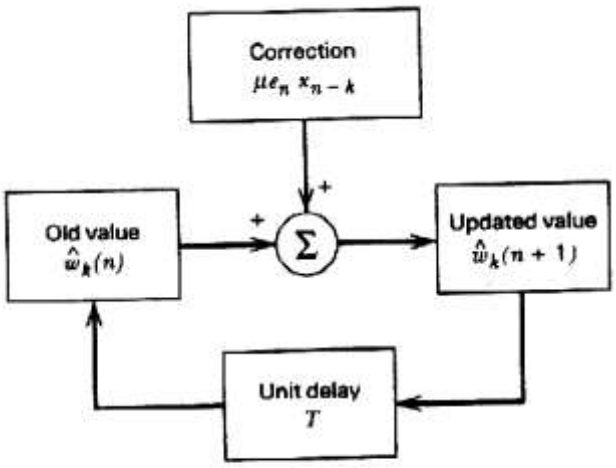


Figure 7.25 Signal-flow graph representation of the LMS algorithm.

Operation of the equalizer:

- square error Training mode
  - Known sequence is transmitted and synchorunized version is generated in the receiver
  - Use the training sequence, so called pseudo-noise(PN) sequence
- Decision-directed mode
  - After training sequence is completed
  - Track relatively slow variation in channel characteristic
- Large μ : fast tracking, excess mean

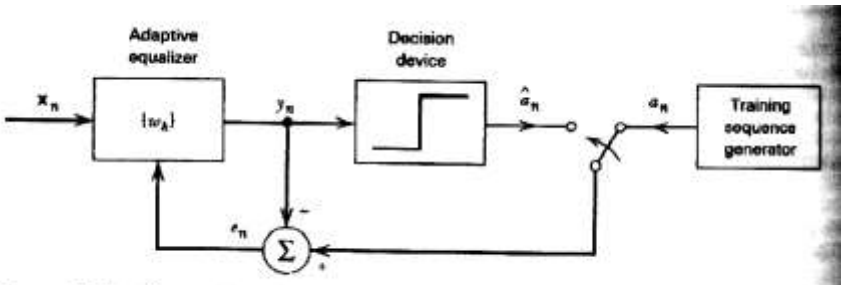


Figure 7.26 Illustrating the two modes of operation of an adaptive equalizer.

Implementation Approaches:

- Analog
  - CCD, Tap-weight is stored in digital memory, analog sample and multiplication
  - Symbol rate is too high
- Digital
  - Sample is quantized and stored in shift register
  - Tap weight is stored in shift register, digital multiplication
- Programmable digital
  - Microprocessor
  - Flexibility
  - Same H/W may be time shared

Decision-Feed back equalization:

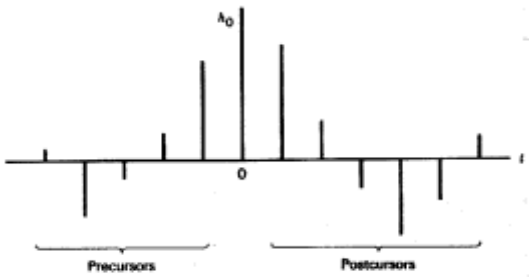


Figure 7.27 Impulse response of a discrete channel.

- Baseband channel impulse response : {hn}, input : {xn}

$$y_n = \sum_k h_k x_{n-k}$$

$$= h_0 x_n + \sum_{k<0} h_k x_{n-k} + \sum_{k>0} h_k x_{n-k}$$

- Using data decisions made on the basis of precursor to take care of the postcursors
  - The decision would obviously have to be correct

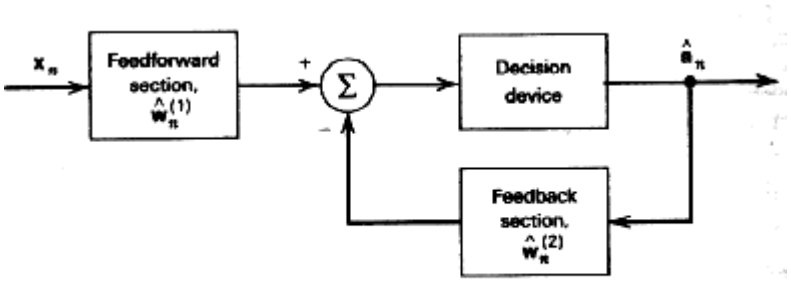


Figure 7.28 Block diagram of decision-feedback equalizer.

- Feedforward section : tapped-delay-line equalizer
- Feedback section : the decision is made on previously detected symbols of the input sequence
  - Nonlinear feedback loop by decision device

$$c_n = \begin{bmatrix} \hat{w}_n^{(1)} \\ \hat{w}_n^{(2)} \end{bmatrix} \qquad v_n = \begin{bmatrix} x_n \\ \hat{a}_n \end{bmatrix} \qquad e_n = a_n - c_n^T v_n \quad \hat{w}_{n+1}^{(1)} = \hat{w}_{n+1}^{(1)} - \mu_1 e_n x_n$$

$$\hat{w}_{n+1}^{(2)} = \hat{w}_{n+1}^{(2)} - \mu_1 e_n \hat{a}_n$$

Eye Pattern:

- Experimental tool for such an evaluation in an insightful manner
  - Synchronized superposition of all the signal of interest viewed within a particular signaling interval
- Eye opening : interior region of the eye pattern

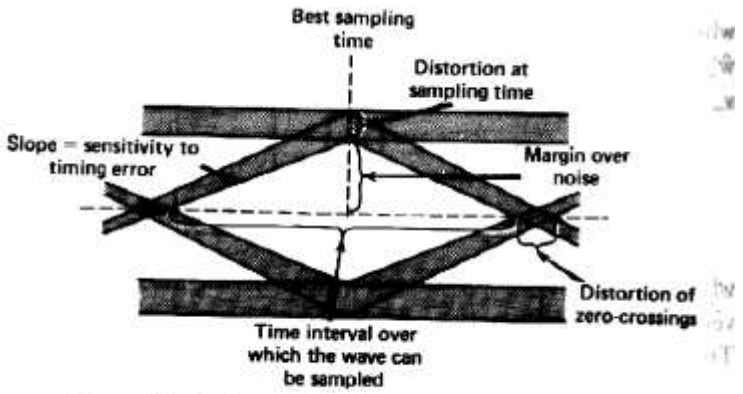
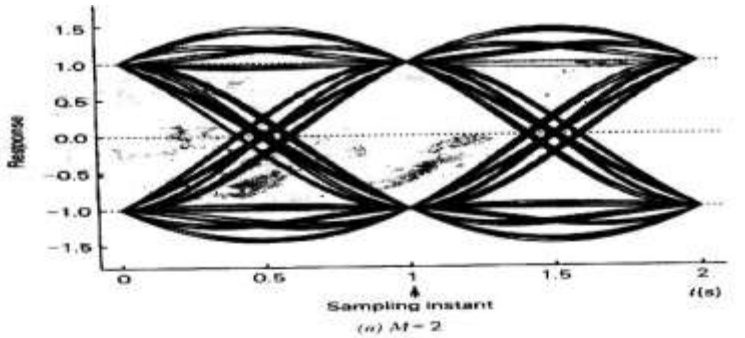
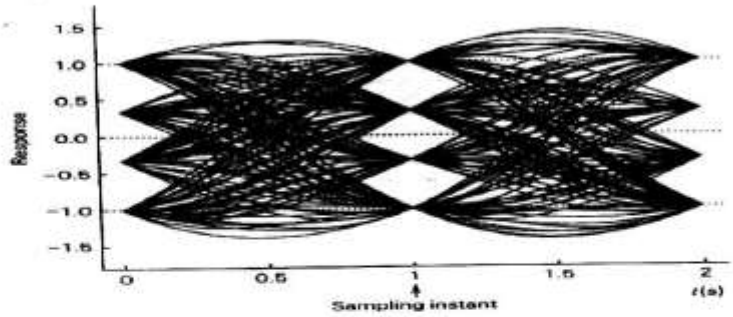


Figure 7.29 Interpretation of the eye pattern.

- In the case of an M-ary system, the eye pattern contains (M-1) eye opening, where M is the number of discreteamplitude levels

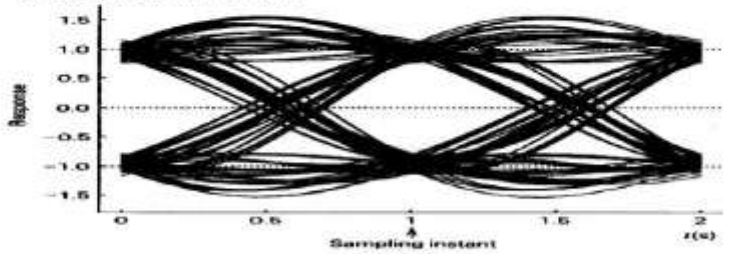


(a) $M = 2$

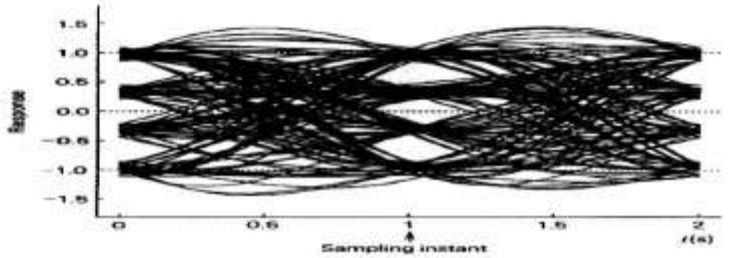


(b) $M = 4$

Figure 7.30 Eye diagram of received signal with no bandwidth limitation.



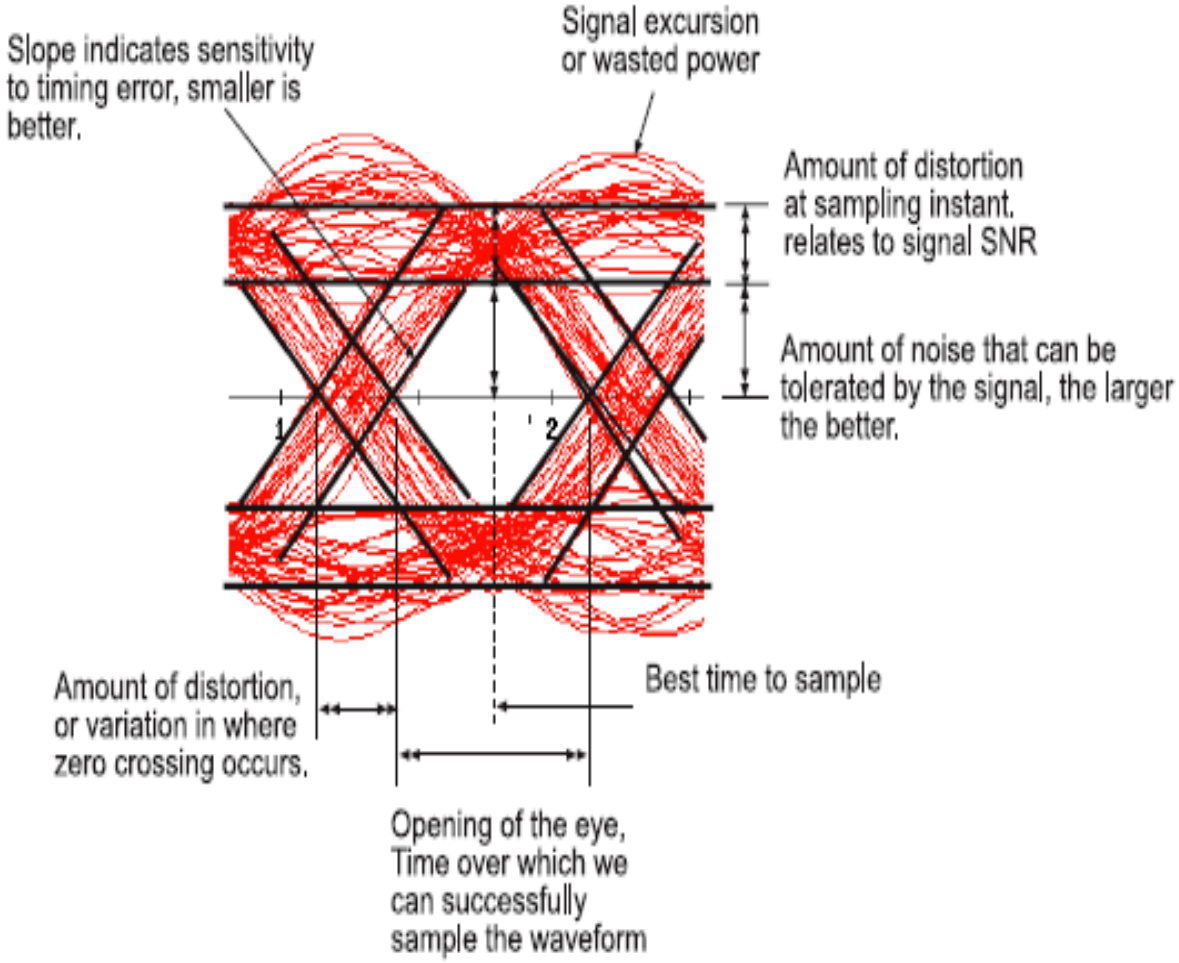
(a) $M = 2$



(b) $M = 4$

Figure 7.31 Eye diagram of received signal, using a bandwidth-limited channel response.

Interpretation of Eye Diagram:


Slope indicates sensitivity to timing error, smaller is better.
Signal excursion or wasted power
Amount of distortion at sampling instant, relates to signal SNR
Amount of noise that can be tolerated by the signal, the larger the better.
1
2
Amount of distortion, or variation in where zero crossing occurs.
Best time to sample
Opening of the eye, Time over which we can successfully sample the waveform

Chapter 6 Passband Data Transmission

6.1 Introduction

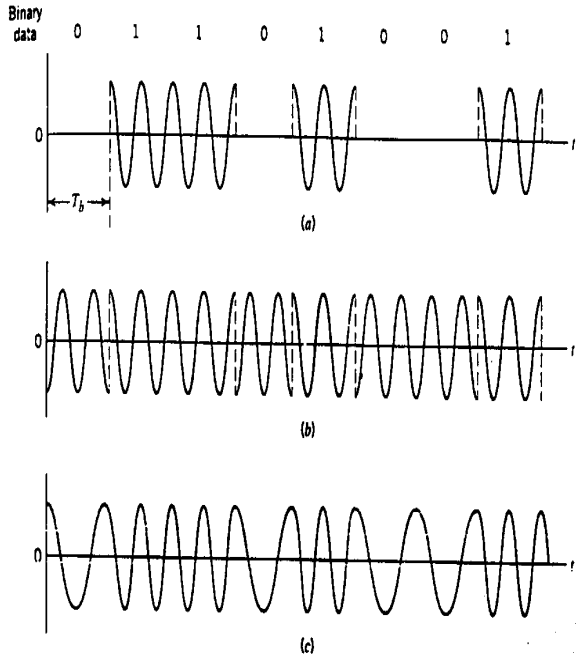


FIGURE 6.1 Illustrative waveforms for the three basic forms of signaling binary information. (*a*) Amplitude-shift keying. (*b*) Phase-shift keying. (*c*) Frequency-shift keying with continuous phase.

Hierarchy of Digital Modulation Techniques

Coherent , Noncoherent

M-ary ASK , M-ary PSK , M-ary FSK

QAM , DPSK

Probability of Error Evaluation

a. exact formulas b. approximate formulas (union bound)

Power Spectra

a.bandwidth

b.cochannel interference in Multiplexing systems

Given a modulated signal $s(t)$

$$s(t) = s_I(t)\cos(2\pi f_c t) - s_Q(t)\sin(2\pi f_c t)$$

$$= \mathrm{Re}\left[\tilde{s}(t)\exp(j2\pi f_c t)\right] \qquad (6.1)$$

The complex envelope is

$$\tilde{s}(t) = s_I(t) + js_Q(t) \qquad (6.2)$$

when $\frac{1}{2}w < f_c$

Let $s_B(f)$ be the baseband power spectral density

The PSD of $s(t)$ is given by

$$S_S(t) = \frac{1}{4}\left[S_B(f - f_c) + S_B(f + f_c)\right] \quad (6.4)$$

(1.55) page 49



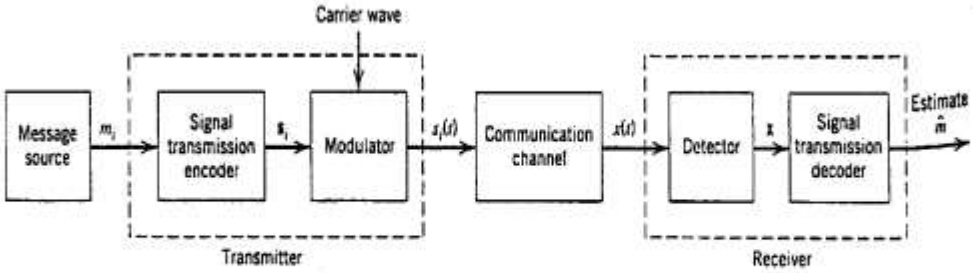


FIGURE 6.2 Functional model of passband data transmission system.

The energy of $s_i(t)$ is

$$E_i = \int_0^T s_i^2(t)dt \, , i = 1,2,..., M \qquad (6.7)$$

Channel Model

a.Linear channel with enough bandwidth , the transmission of $s_i(t)$ is no distortion

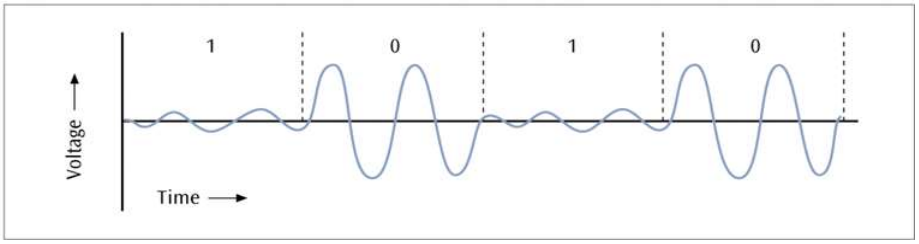
b.AWGN with zero mean and PSD $N_0/2$

The reciever recovers the original message by computing the estimate $\hat{m}$ with minimum noise effect

ASK, OOK, MASK:

- The amplitude (or height) of the sine wave varies to transmit the ones and zeros

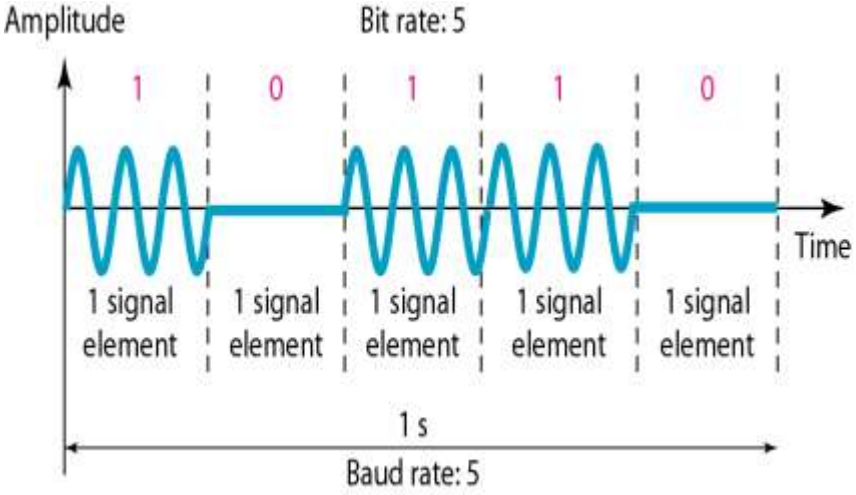
Figure 2-15
Example of amplitude shift keying

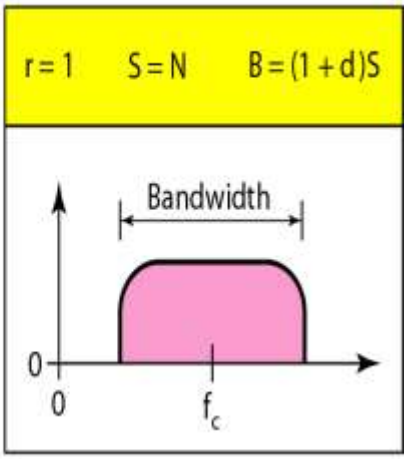


- One amplitude encodes a 0 while another amplitude encodes a 1 (a form of amplitude modulation)

Binary amplitude shift keying, Bandwidth:

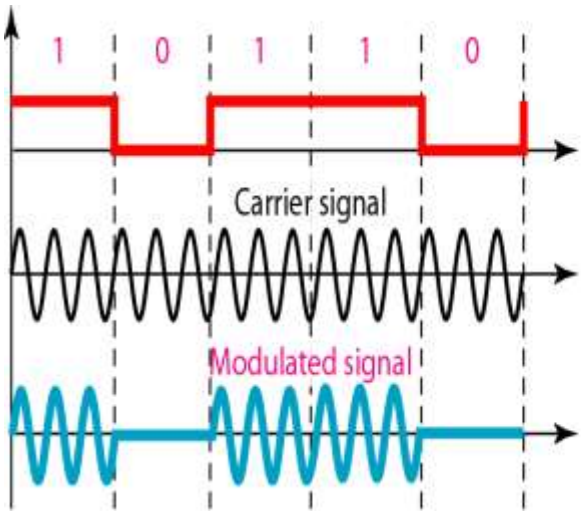
- $d \geq 0$-related to the condition of the line

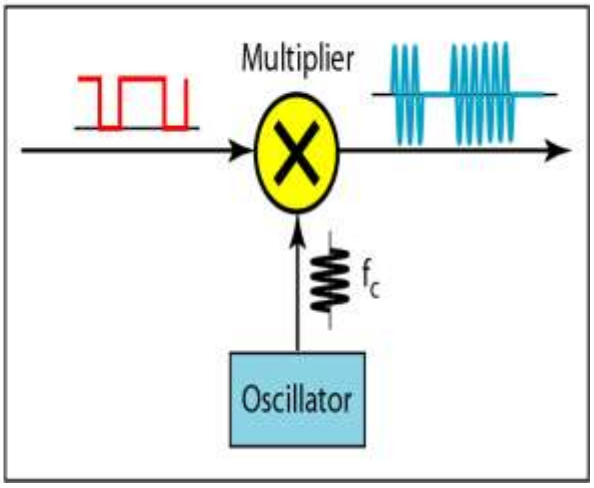




B = (1+d) x S = (1+d) x N x 1/r

Implementation of binary ASK:

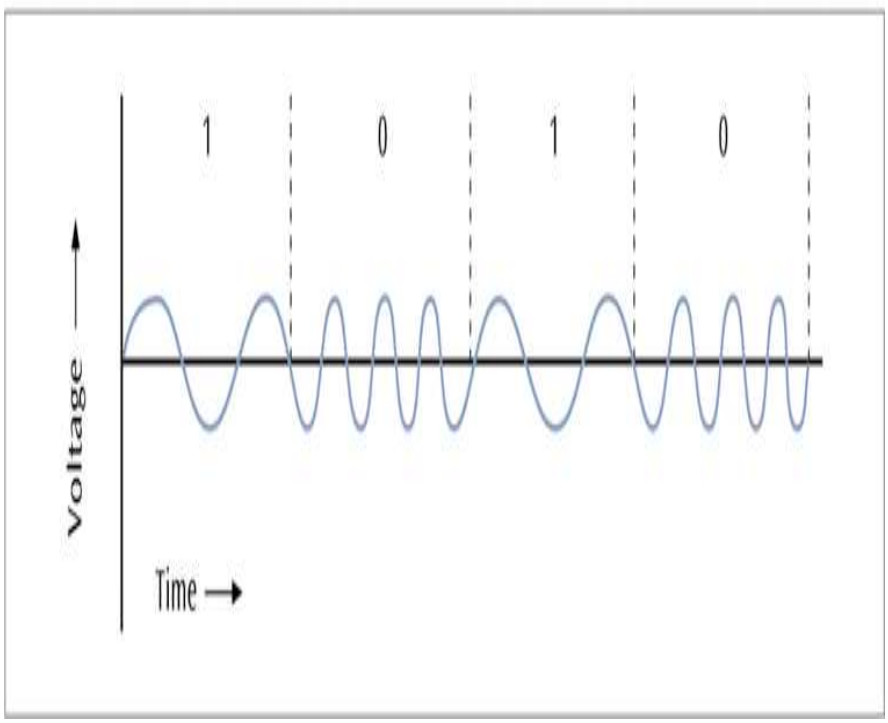




Frequency Shift Keying:

- One frequency encodes a 0 while another frequency encodes a 1 (a form of frequency modulation)

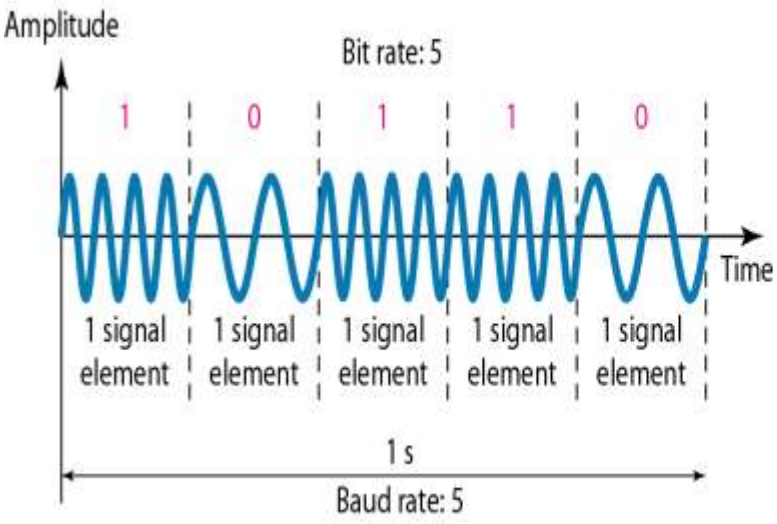
Figure 2-17
Simple example of frequency shift keying

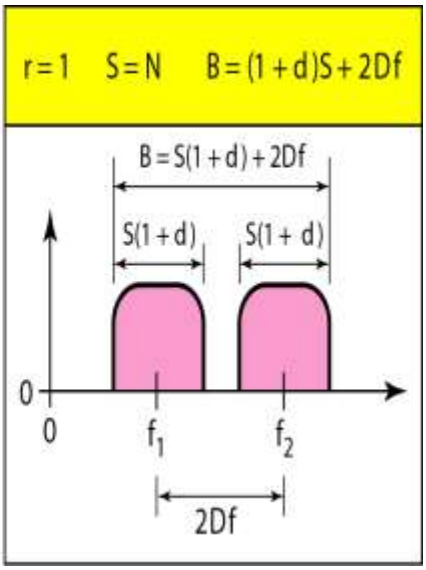


$$s(t) = \begin{cases} A\cos(2\pi f_2 t) & \text{binary 1} \\ A\cos(2\pi f_2 t) & \text{binary 0} \end{cases}$$

FSK Bandwidth:

- Limiting factor: Physical capabilities of the carrier
- Not susceptible to noise as much as ASK

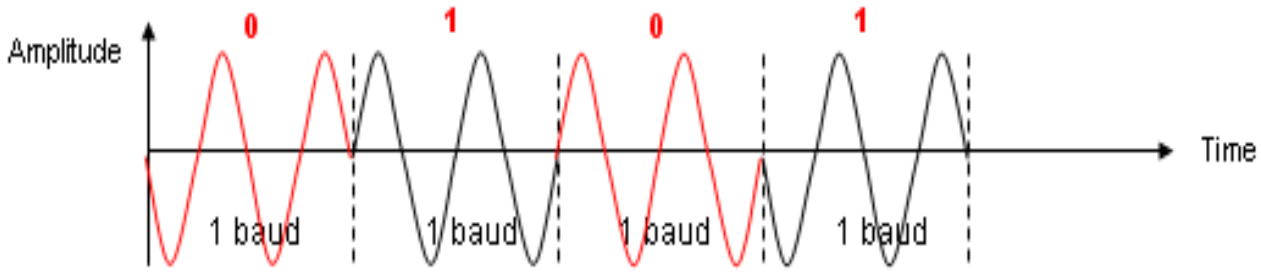




- Applications
  - On voice-grade lines, used up to 1200bps
  - Used for high-frequency (3 to 30 MHz) radio transmission
  - used at higher frequencies on LANs that use coaxial cable

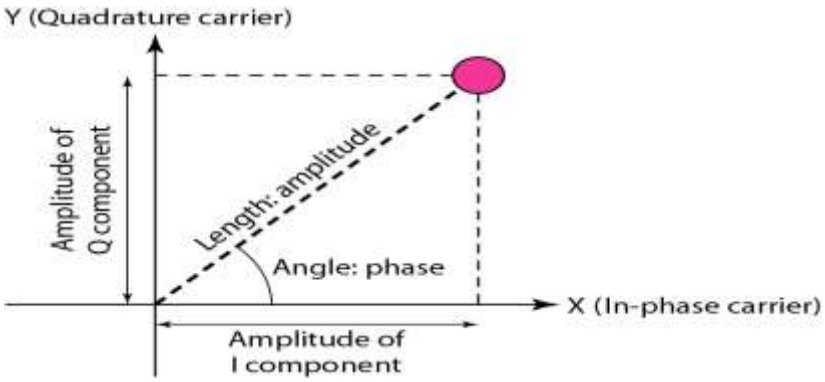
DBPSK:

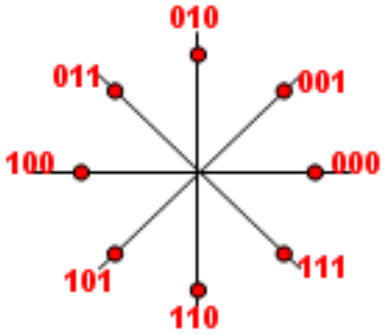
- Differential BPSK
  - 0 = same phase as last signal element
  - 1 = 180º shift from last signal element



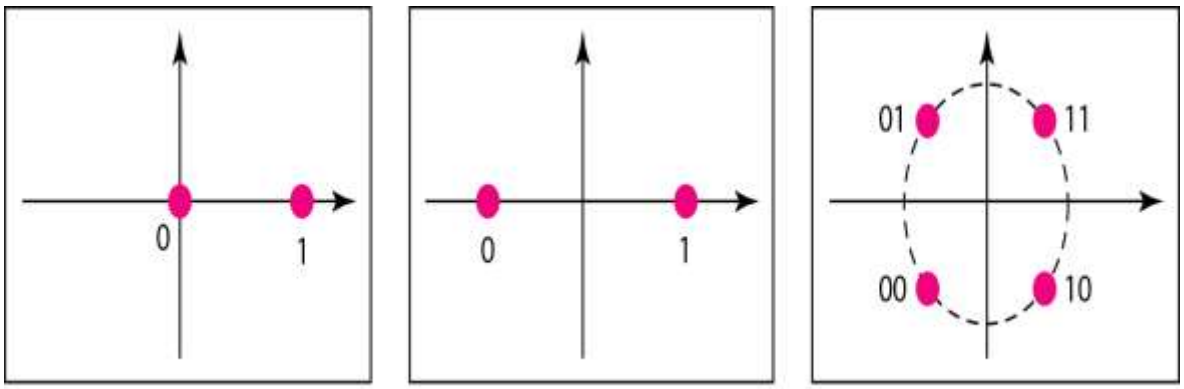
$$s(t) = \begin{cases} A\cos\left(2\pi f_c t + \dfrac{\pi}{4}\right) & 11 \\ A\cos\left(2\pi f_c t + \dfrac{3\pi}{4}\right) & 01 \\ A\cos\left(2\pi f_c t - \dfrac{3\pi}{4}\right) & 00 \\ A\cos\left(2\pi f_c t - \dfrac{\pi}{4}\right) & 10 \end{cases}$$

Concept of a constellation :





8-PSK



a. ASK (OOK) b. BPSK c. QPSK

M-ary PSK:

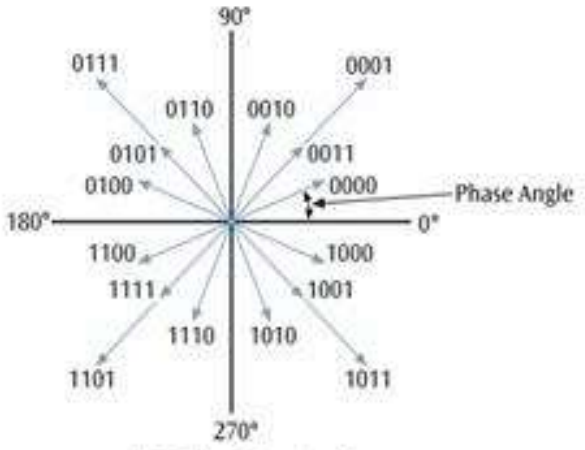
Using multiple phase angles with each angle having more than one amplitude, multiple signals elements can be achieved

$$D = \frac{R}{L} = \frac{R}{\log_2 M}$$

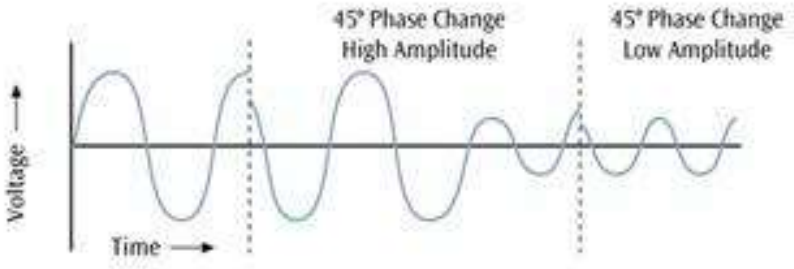
- D = modulation rate, baud
- R = data rate, bps
- M = number of different signal elements = $2L$
- L = number of bits per signal element

QAM:

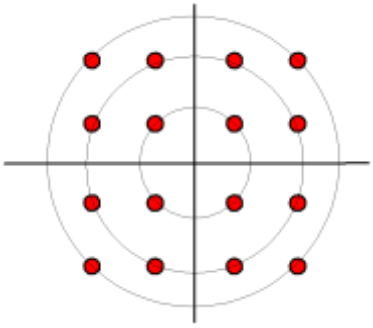
- As an example of QAM, 12 different phases are combined with two different amplitudes
- Since only 4 phase angles have 2 different amplitudes, there are a total of 16 combinations
- With 16 signal combinations, each baud equals 4 bits of information (2 ^ 4 = 16)
- Combine ASK and PSK such that each signal corresponds to multiple bits
- More phases than amplitudes
- Minimum bandwidth requirement same as ASK or PSK



(a) Twelve Phase Angles

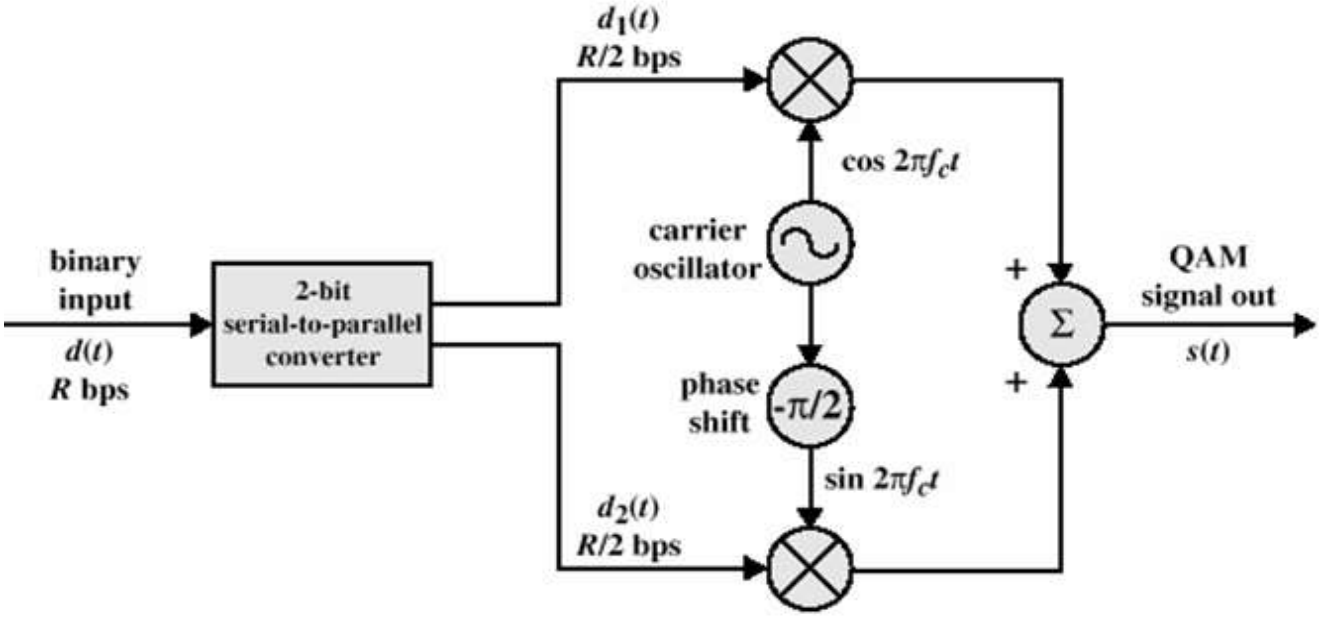


(b) A Phase Change with Two Amplitudes



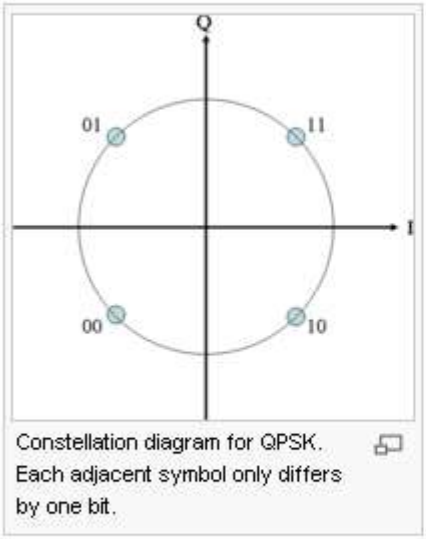
QAM and QPR:

- QAM is a combination of ASK and PSK
  - Two different signals sent simultaneously on the same carrier frequency
  - M=4, 16, 32, 64, 128, 256
- Quadrature Partial Response (QPR)
  - 3 levels (+1, 0, -1), so 9QPR, 49QPR

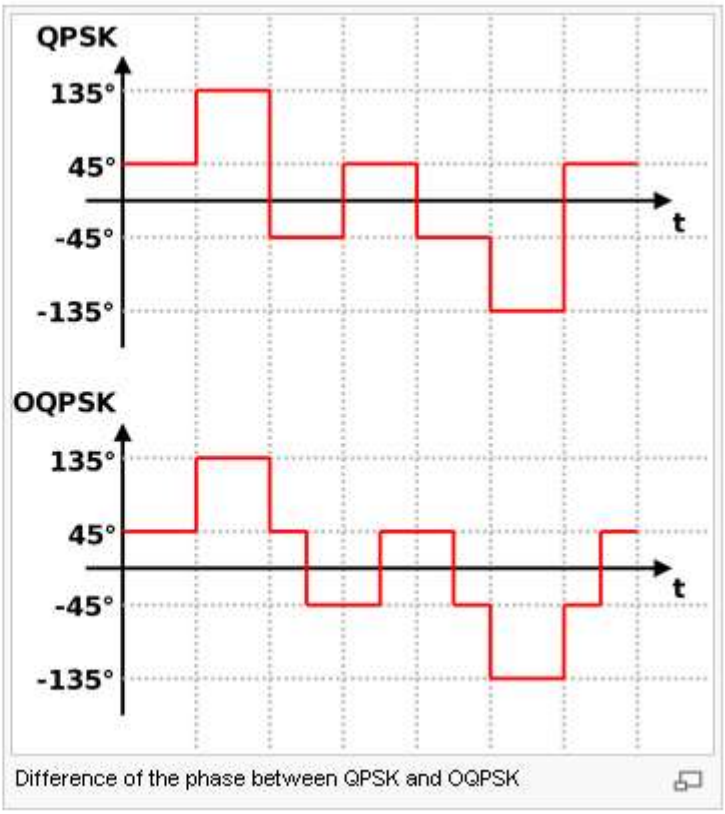


Offset quadrature phase-shift keying (OQPSK):

- QPSK can have 180 degree jump, amplitude fluctuation
- By offsetting the timing of the odd and even bits by one bit-period, or half a symbol-period, the in-phase and quadrature components will never change at the same time.



Constellation diagram for QPSK. Each adjacent symbol only differs by one bit.



Difference of the phase between QPSK and OQPSK

Generation and Detection of Coherent BPSK:

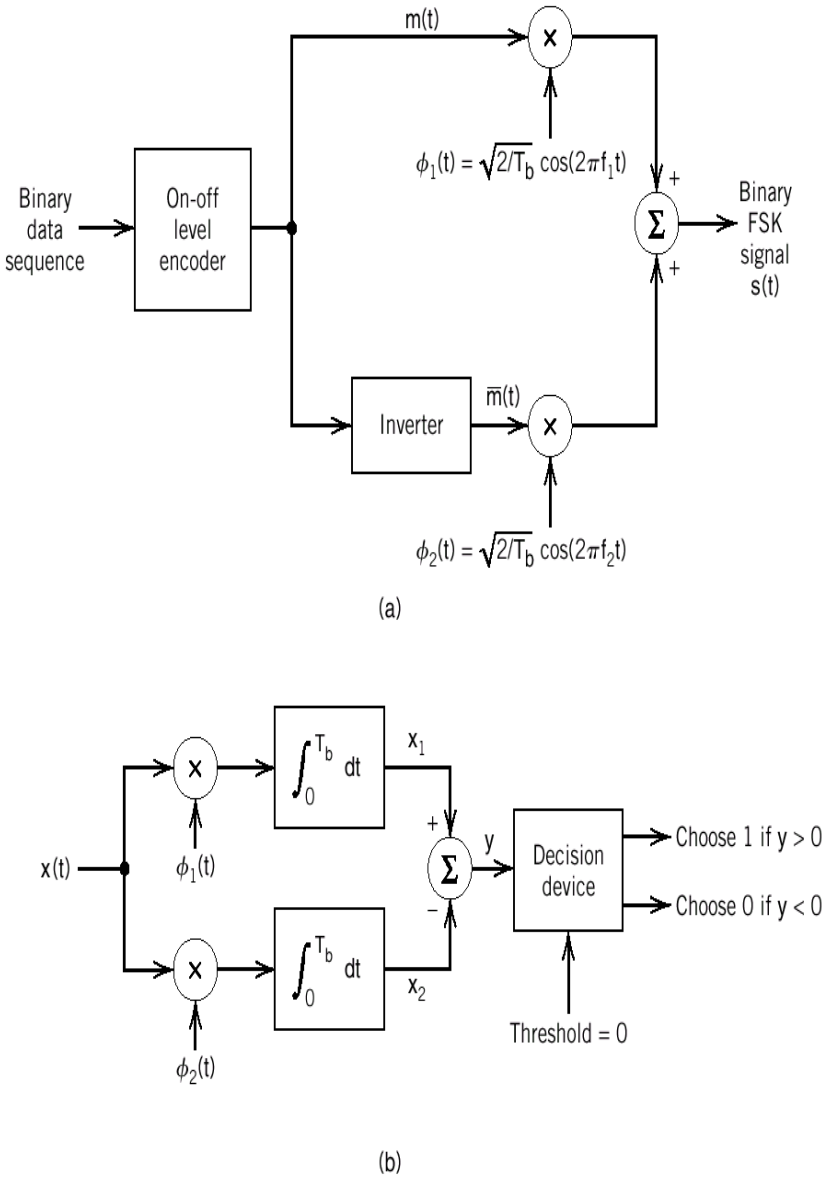


Figure 6.26 ***Block diagrams for (a) binary FSK transmitter and (b) coherent binary FSK receiver.***

Fig. 6.28

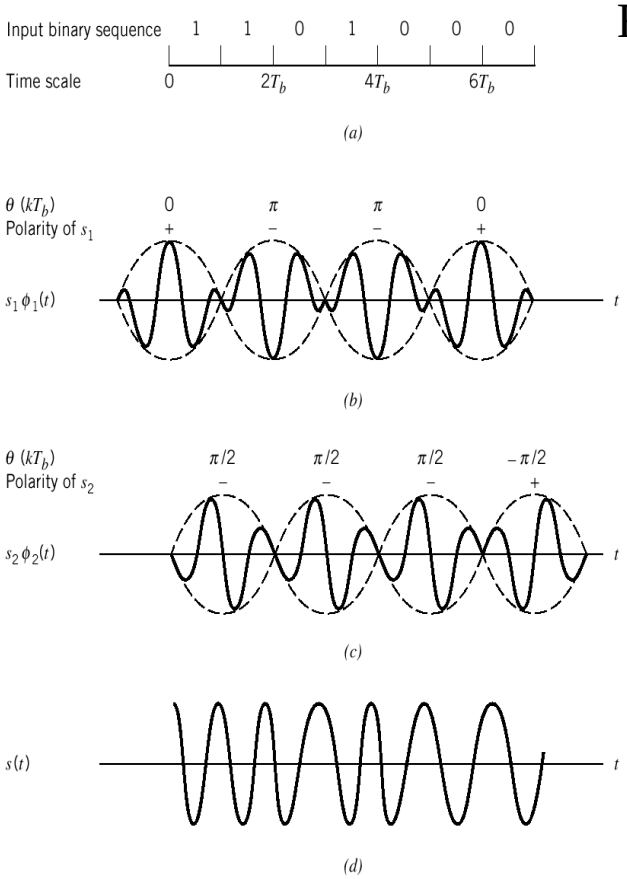


Figure 6.30 ***(a) Input binary sequence. (b) Waveform of scaled timefunction $s_1f_1(t)$. (c) Waveform of scaled time function $s_2f_2(t)$. (d) Waveform of the MSK signal s(t) obtained by adding $s_1f_1(t)$ and***

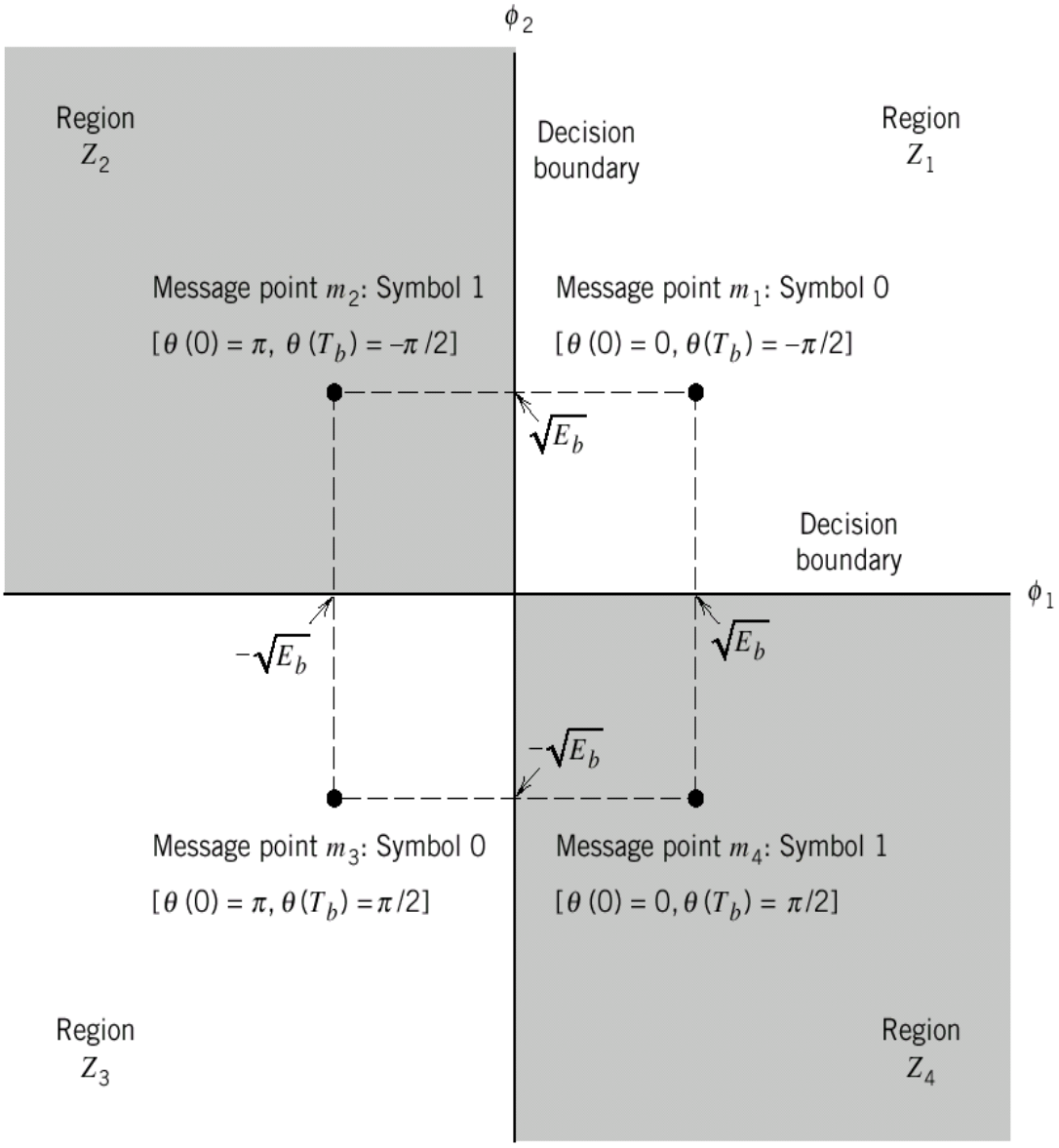


Figure 6.29 ***Signal-space diagram for MSK system.***

Generation and Detection of MSK Signals:

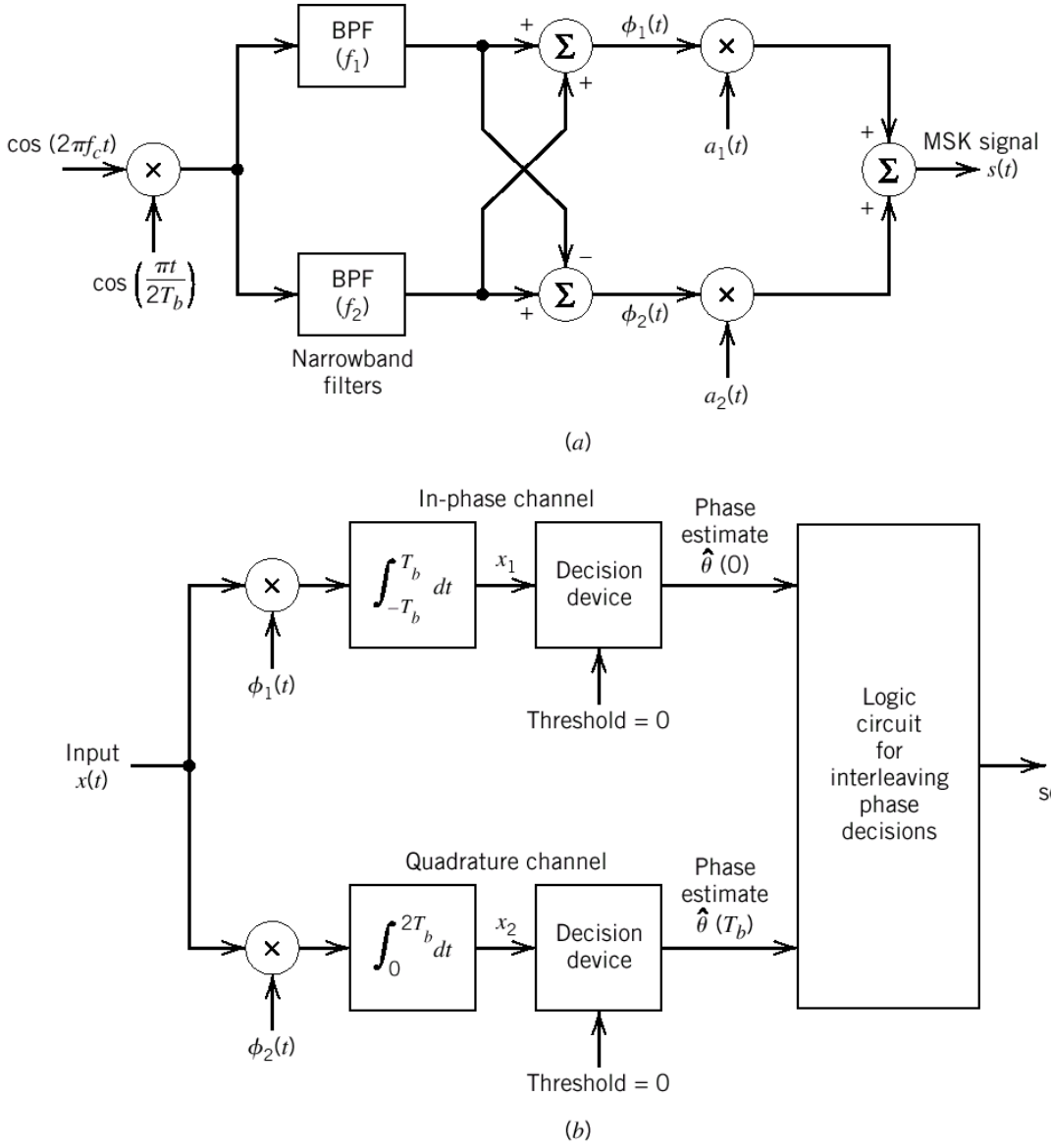


Figure 6.31 ***Block diagrams for (a) MSK transmitter and (b) coherent MSK receiver.***

Power Spectra of MSK

1.The in - phase component is (6.118)

$$g(t)=\pm\sqrt{\frac{2E_b}{T_b}}\cos(\frac{\pi t}{2T_b})\ ,\quad -T_b\le t\le T_b \qquad (6.128)$$

$$g(t)\leftrightarrow\frac{4\sqrt{2E_bT_b}}{\pi}\frac{\cos(2\pi T_bf)}{1-(4T_bf)^2}$$

The energy spectra of $g(t)$

$$\psi_g(f)=\frac{32E_bT_b}{\pi^2}\left[\frac{\cos(2\pi T_bf)}{1-(4T_bf)^2}\right]^2$$

2.The quadrature component (6.119)

$$g'(t)=\sqrt{\frac{2E_b}{T_b}}\sin(\frac{\pi t}{2T_b})\ ,\quad 0\le t\le 2T_b \qquad (6.130)$$

$$\psi_g'(f)=\psi_g(f)$$

The PSD of MSK

$$S_B(f)=\frac{\psi_g(f)}{2T_b}+\frac{\psi_g'(f)}{2T_b}$$

$$=2\left[\frac{\psi_g(f)}{2T_b}\right]$$

$$=\frac{32E_b}{\pi^2}\left[\frac{\cos(2\pi T_bf)}{16T_b^2f^2-1}\right]^2 \qquad (6.131)$$

1.Constant envelope

2.Relatively narrow bandwidth

3.Performance ~ QPSK

However the out - of - band spectra may interfer the adjacent channel in wireless communication ⇒ Gaussian - Filtered MSK (used in GSM)

Fig 6.9

M - ary FSK

$$s_i(t) = \sqrt{\frac{2E}{T}} \cos\left[\frac{\pi}{T}(n_c + i)t\right], \quad 0 \le t \le T \qquad (6.137)$$

$i = 1,2,...,M$

T : symbol duration

E : symbol energy

$$\int_0^T s_i(t)s_j(t)dt = \delta_{ij} \qquad (6.138)$$

$$\phi_i(t) = \frac{1}{\sqrt{E}} s_i(t), \quad 0 \le t \le T, \; i = 1,2,..,M \qquad (6.139)$$

$$d_{min} = \sqrt{2E}$$

Recall (5.95, page 335)

$$P_e \le \frac{(M-1)}{2} erfc\left(\frac{d_{min}}{2\sqrt{N_0}}\right) = \frac{M-1}{2} erfc\left(\sqrt{\frac{E}{2N_0}}\right) \qquad (6.140)$$

- **Forward Error Correction (FEC)**
  - Coding designed so that errors can be corrected at the receiver
  - Appropriate for delay sensitive and one-way transmission (e.g., broadcast TV) of data
  - Two main types, namely block codes and convolutional codes. We will only look at block codes

Block Codes:

- We will consider only binary data
- Data is grouped into blocks of length *k* bits (dataword)
- Each dataword is coded into blocks of length *n* bits (codeword), where in general $n>k$
- This is known as an (*n*,*k*) block code
- A vector notation is used for the datawords and codewords,
  - Dataword d = (*d1 d2….dk*)
  - Codeword c = (*c1 c2……..cn*)
- The redundancy introduced by the code is quantified by the code rate,
  - Code rate = *k*/*n*
  - i.e., the higher the redundancy, the lower the code rate

Hamming Distance:

- Error control capability is determined by the Hamming distance
- The Hamming distance between two codewords is equal to the number of differences between them, e.g.,

10011011
11010010 have a Hamming distance = 3

- Alternatively, can compute by adding codewords (mod 2) =01001001 (now count up the ones)

- The maximum number of detectable errors is

$$d_{\min} - 1$$

- That is the maximum number of correctable errors is given by,

$$t = \left| \frac{d_{\min} - 1}{2} \right|$$

where *d*min is the minimum Hamming distance between 2 codewords and means the smallest integer

Linear Block Codes:

- As seen from the second Parity Code example, it is possible to use a table to hold all the codewords for a code and to look-up the appropriate codeword based on the supplied dataword
- Alternatively, it is possible to create codewords by addition of other codewords. This has the advantage that there is now no longer the need to held every possible codeword in the table.
- If there are *k* data bits, all that is required is to hold *k* linearly independent codewords, i.e., a set of *k* codewords none of which can be produced by linear combinations of 2 or more codewords in the set.
- The easiest way to find *k* linearly independent codewords is to choose those which have „1" in just one of the first *k* positions and „0" in the other *k*-1 of the first *k* positions.

- **For example for a (7,4) code, only four codewords are required, e.g.,**

1	0	0	0	1	1	0
0	1	0	0	1	0	1
0	0	1	0	0	1	1
0	0	0	1	1	1	1

- So, to obtain the codeword for dataword 1011, the first, third and fourth codewords in the list are added together, giving 1011010
- This process will now be described in more detail

- An (*n*,*k*) block code has code vectors

d=(*d1 d2*….*dk*) and
c=(*c1 c2*…….*cn*)

- The block coding process can be written as c=dG
  where G is the Generator Matrix

$$G = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ . & . & \dots & . \\ a_{k1} & a_{k2} & \dots & a_{kn} \end{bmatrix} = \begin{bmatrix} a_1 \\ a_2 \\ . \\ a_k \end{bmatrix}$$

- Thus,

$$c = \sum_{i=1}^{k} d_i a_i$$

- a_i must be linearly independent, i.e.,
  Since codewords are given by summations of the ai vectors, then to avoid 2 datawords having the same codeword the ai vectors must be linearly independent.
- Sum (mod 2) of any 2 codewords is also a codeword, i.e.,
  Since for datawords d1 and d2 we have;

$$d_3 = d_1 + d_2$$

So,

$$c_3 = \sum_{i=1}^{k} d_{3i} a_i = \sum_{i=1}^{k} (d_{1i} + d_{2i}) a_i = \sum_{i=1}^{k} d_{1i} a_i + \sum_{i=1}^{k} d_{2i} a_i$$

$$c_3 = c_1 + c_2$$

Error Correcting Power of LBC:

- The Hamming distance of a linear block code (LBC) is simply the minimum Hamming weight (number of 1‟s or equivalently the distance from the all 0 codeword) of the non-zero codewords
- Note d(c1,c2) = w(c1+ c2) as shown previously
- For an LBC, c1+ c2=c3
- So min (d(c1,c2)) = min (w(c1+ c2)) = min (w(c3))
- Therefore to find min Hamming distance just need to search among the 2*k* codewords to find the min Hamming weight – far simpler than doing a pair wise check for all possible codewords.
-

Linear Block Codes – example 1:

- For example a (4,2) code, suppose;

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

a1 = [1011]
a2 = [0101]

- For d = [1 1], then;

$$c = \begin{array}{rcccc} & 1 & 0 & 1 & 1 \\ + & 0 & 1 & 0 & 1 \\ \hline = & 1 & 1 & 1 & 0 \end{array}$$

Linear Block Codes – example 2:

- A (6,5) code with

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

- Is an even single parity code

Systematic Codes:

- For a systematic block code the dataword appears unaltered in the codeword – usually at the start
- The generator matrix has the structure,

$$G = \begin{bmatrix} 1 & 0 & .. & 0 & p_{11} & p_{12} & .. & p_{1R} \\ 0 & 1 & .. & 0 & p_{21} & p_{22} & .. & p_{2R} \\ .. & .. & .. & .. & .. & .. & .. & .. \\ 0 & 0 & .. & 1 & p_{k1} & p_{k2} & .. & p_{kR} \end{bmatrix} = [\mathrm{I} \mid \mathrm{P}] \qquad R = n - k$$

- P is often referred to as parity bits

I is *k***k* identity matrix. Ensures data word appears as beginning of codeword P is *k***R* matrix.

Decoding Linear Codes:

- One possibility is a ROM look-up table
- In this case received codeword is used as an address
- Example – Even single parity check code;

Address	Data
000000	0
000001	1
000010	1
000011	0
……	.

- Data output is the error flag, i.e., 0 – codeword ok,
- If no error, data word is first *k* bits of codeword
- For an error correcting code the ROM can also store data words

- Another possibility is algebraic decoding, i.e., the error flag is computed from the received codeword (as in the case of simple parity codes)
- How can this method be extended to more complex error detection and correction codes?

Parity Check Matrix:

- A linear block code is a linear subspace S_{sub} of all length *n* vectors (Space S)
- Consider the subset S null of all length *n* vectors in space S that are orthogonal to all length *n* vectors in S_{sub}
- It can be shown that the dimensionality of S_{null} is *n-k*, where *n* is the dimensionality of S and *k* is the dimensionality of S_{sub}
- It can also be shown that S_{null} is a valid subspace of S and consequently S_{sub} is also the null space **of** S_{null}

- S_{null} can be represented by its basis vectors. In this case the generator basis vectors (or „generator matrix" H) denote the generator matrix for S_{null} - of dimension $n-k = R$
- This matrix is called the *parity check matrix* of the code defined by G, where G is obviously the generator matrix for S_{sub} - of dimension *k*
- Note that the number of vectors in the basis defines the dimension of the subspace
- So the dimension of H is $n-k$ (= R) and all vectors in the null space are orthogonal to all the vectors of the code
- Since the rows of H, namely the vectors bi are members of the null space they are orthogonal to any code vector
- So a vector y is a codeword only if yHT=0
- Note that a linear block code can be specified by either G or H

Parity Check Matrix:

$$H = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1n} \\ b_{21} & b_{22} & \dots & b_{2n} \\ . & . & \dots & . \\ b_{R1} & b_{R2} & \dots & b_{Rn} \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ . \\ b_R \end{bmatrix} \quad R = n - k$$

- So H is used to check if a codeword is valid,
- The rows of H, namely, bi, are chosen to be orthogonal to rows of G, namely ai
- Consequently the dot product of any valid codeword with any bi is zero

This is so since,

$$c = \sum_{i=1}^{k} d_i a_i$$

and so,

$$b_j.c = b_j.\sum_{i=1}^{k} d_i a_i = \sum_{i=1}^{k} d_i (a_i.b_j) = 0$$

- This means that a codeword is valid (but not necessarily correct) only if cHT = 0. To ensure this it is required that the rows of H are independent and are orthogonal to the rows of G
- That is the bi span the remaining R (= n - k) dimensions of the codespace

- For example consider a (3,2) code. In this case G has 2 rows, a1 and a2
- Consequently all valid codewords sit in the subspace (in this case a plane) spanned by a1 and a2

- In this example the H matrix has only one row, namely b1. This vector is orthogonal to the plane containing the rows of the G matrix, i.e., a1 and a2
- Any received codeword which is not in the plane containing a1 and a2 (i.e., an invalid codeword) will thus have a component in the direction of b1 yielding a non- zero dot product between itself and b1.

Error Syndrome:

- For error correcting codes we need a method to compute the required correction
- To do this we use the Error Syndrome, s of a received codeword, cr

  $$s = crHT$$
- If cr is corrupted by the addition of an error vector, e, then

  $$cr = c + e$$

  and

  $$s = (c + e)\,HT = cHT + eHT$$

  $$s = 0 + eHT$$

  Syndrome depends only on the error
- That is, we can add the same error pattern to different code words and get the same syndrome.
  - There are $2(n - k)$ syndromes but $2n$ error patterns
  - For example for a (3,2) code there are 2 syndromes and 8 error patterns
  - Clearly no error correction possible in this case
  - Another example. A (7,4) code has 8 syndromes and 128 error patterns.
  - With 8 syndromes we can provide a different value to indicate single errors in any of the 7 bit positions as well as the zero value to indicate no errors
- Now need to determine which error pattern caused the syndrome

- For systematic linear block codes, H is constructed as follows,

  G = [I | P] and so H = [-PT | I]

  where I is the *k***k* identity for G and the *R***R* identity for H
- Example, (7,4) code, *dmin*= 3

$$G = [I \mid P] = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix} \qquad H = [-P^T \mid I] = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Error Syndrome – Example:

- For a correct received codeword cr = [1101001]
  In this case,

$$s = c_r H^T = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$$

Standard Array:

- The Standard Array is constructed as follows,

c_1 (all zero)	c_2	……	c_M	s_0
e_1	c_2+e_1	……	c_M+e_1	s_1
e_2	c_2+e_2	……	c_M+e_2	s_2
e_3	$\mathbf{c_2+e_3}$	……	c_M+e_3	s_3
…	……	……	……	…
e_N	c_2+e_N	……	c_M+e_N	s_N

- The array has $2k$ columns (i.e., equal to the number of valid codewords) and $2R$ rows (i.e., the number of syndromes)

Hamming Codes:

- We will consider a special class of SEC codes (i.e., Hamming distance = 3) where,
  - Number of parity bits R = $n - k$ and n = $2R - 1$
  - Syndrome has R bits
  - 0 value implies zero errors
  - $2R - 1$ other syndrome values, i.e., one for each bit that might need to be corrected
  - This is achieved if each column of H is a different binary word – remember s = eHT
- Systematic form of (7,4) Hamming code is,

$$G = [I \mid P] = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix} \qquad H = [-P^T \mid I] = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

- The original form is non-systematic,

$$G = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix} \qquad H = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{bmatrix}$$

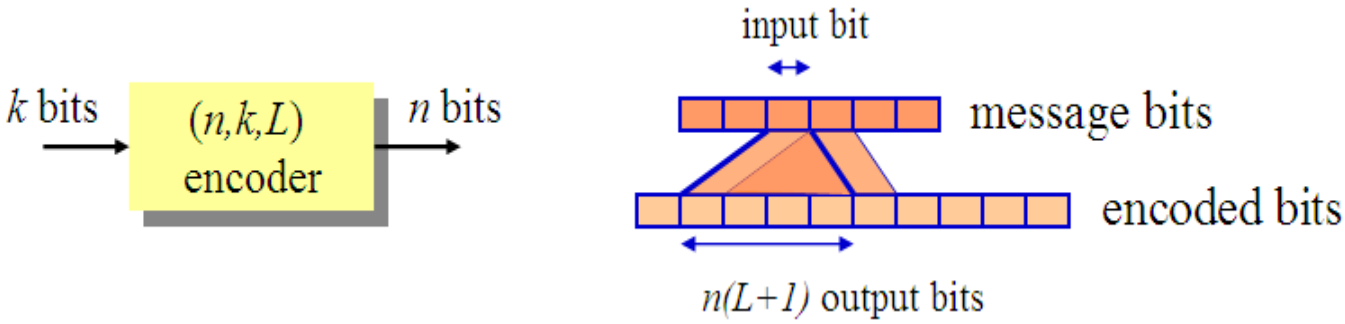
- Compared with the systematic code, the column orders of both G and H are swapped so that the columns of H are a binary count
- The column order is now 7, 6, 1, 5, 2, 3, 4, i.e., col. 1 in the non-systematic H is col. 7 in the systematic H.

Convolutional Code Introduction:

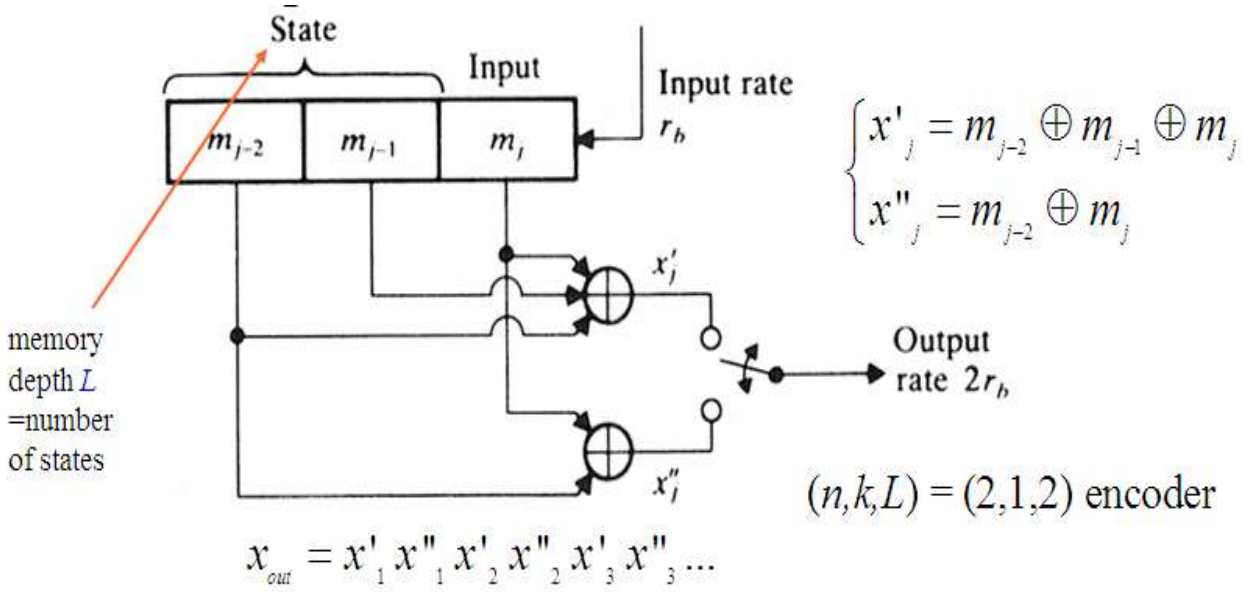
- Convolutional codes map information to code bits sequentially by convolving a sequence of information bits with "generator" sequences
- A convolutional encoder encodes *K* information bits to *N*>*K* code bits at one time step
- Convolutional codes can be regarded as block codes for which the encoder has a certain structure such that we can express the encoding operation as convolution

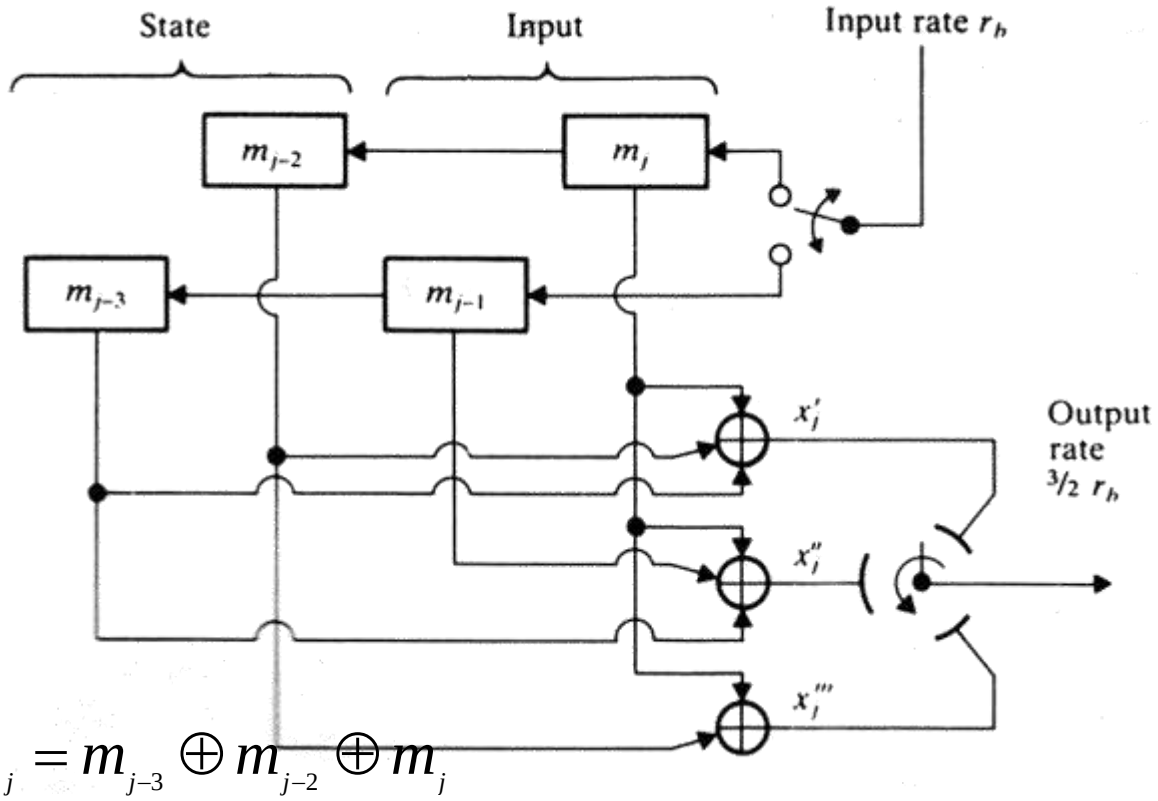
- Convolutional codes are applied in applications that require good performance with low implementation cost. They operate on code streams (not in blocks)
- Convolution codes have memory that utilizes previous bits to encode or decode following bits (block codes are memoryless)
- Convolutional codes achieve good performance by expanding their memory depth
- Convolutional codes are denoted by (*n*,*k*,*L*), where ***L*** is code (or encoder) **Memory depth** (number of register stages)

- **Constraint length *C=n(L+1)*** is defined as the number of encoded bits a message bit can influence to



- Convolutional encoder, $k = 1$, $n = 2$, L=2
  - Convolutional encoder is a **finite state machine** (FSM) processing information bits in a serial manner
  - Thus the generated code is a function of input and the state of the FSM
  - In this $(n,k,L) = (2,1,2)$ encoder each message bit influences a span of $C = n(L+1)=6$ successive output bits = **constraint length *C***
  - Thus, for generation of n-bit output, we require n shift registers in $k = 1$ convolutional encoders





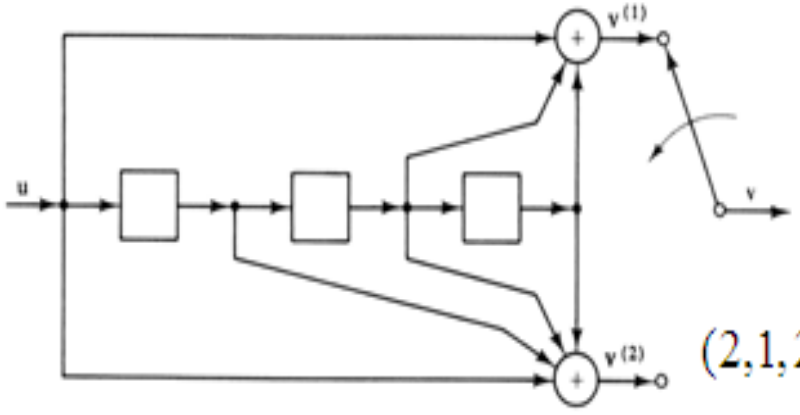
$$x'_j = m_{j-3} \oplus m_{j-2} \oplus m_j$$

$$x''_j = m_{j-3} \oplus m_{j-1} \oplus m_j$$

$$x'''_j = m_{j-2} \oplus m_j$$

Here each message bit influences a span of $C = n(L+1)=3(1+1)=6$ successive output bits

- (n,k,L) Convolutional code can be described by the generator sequences $\mathbf{g}^{(1)}, \mathbf{g}^{(2)}, \ldots \mathbf{g}^{(n)}$ that are the impulse responses for each coder n output branches:



(2,1,2) encoder

$$\begin{cases} \mathbf{g}^{(1)} = [1\ 0\ 1\ 1] \\ \mathbf{g}^{(2)} = [1\ 1\ 1\ 1] \end{cases}$$

Note that the generator sequence length exceeds register depth always by 1

- Generator sequences specify convolutional code completely by the associated generator matrix
- Encoded convolution code is produced by matrix multiplication of the input and the generator matrix

Convolution point of view in encoding and generator matrix:

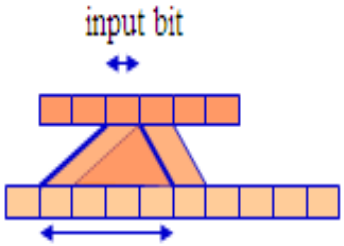
- Encoder outputs are formed by **modulo-2 discrete convolutions:**

$$x \otimes y(u) = \sum_{k \in A_y} x(k)\, y(u-k)$$

$$\mathbf{v}^{(1)} = \mathbf{u} * \mathbf{g}^{(1)}, \mathbf{v}^{(2)} = \mathbf{u} * \mathbf{g}^{(2)} \ldots \mathbf{v}^{(j)} = \mathbf{u} * \mathbf{g}^{(j)}$$

where $\mathbf{u}$ is the information sequence:

$$\mathbf{u} = (u_0, u_1, \cdots)$$



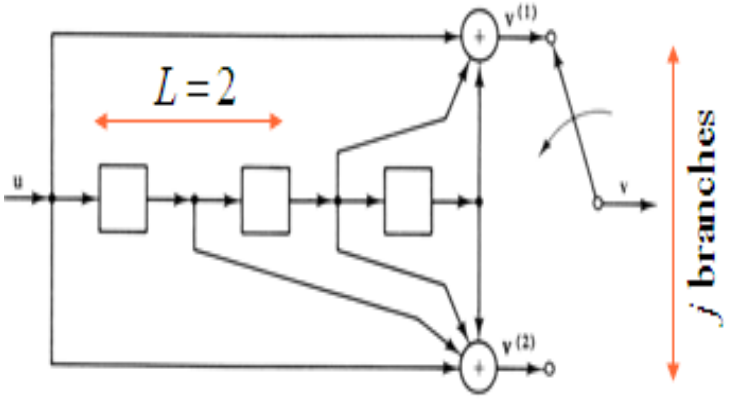
- Therefore, the l:th bit of the j:th output branch is*

$$v_l^{(j)} = \sum_{i=0}^{m} u_{l-i} g_i^{(j)} = u_l g_0^{(j)} + u_{l-1} g_1^{(j)} + \ldots + u_{l-m} g_m^{(j)}$$

where $m = L+1, u_{l-i} \square\, 0, l < i$

- Hence, for this circuit the following equations result, (assume:

$$\begin{cases} \mathbf{g}^{(1)} = [1\ 0\ 1\ 1] \\ \mathbf{g}^{(2)} = [1\ 1\ 1\ 1] \end{cases}$$

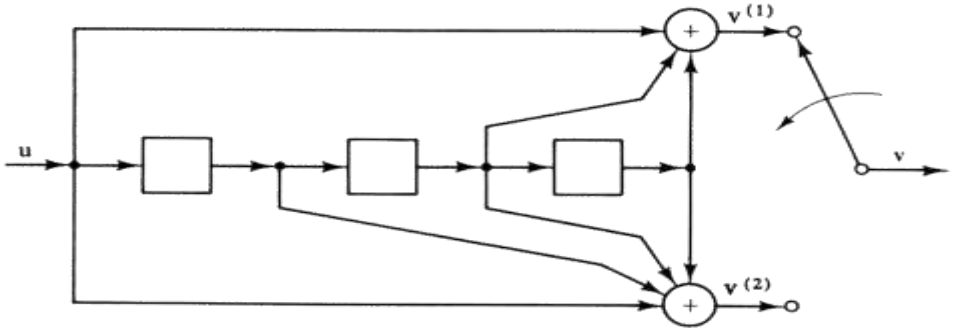


$$\begin{cases} v_l^{(1)} = u_l + u_{l-2} + u_{l-3} \\ v_l^{(2)} = u_l + u_{l-1} + u_{l-2} + u_{l-3} \end{cases}$$

encoder output:

$$\mathbf{v} = [v_0^{(1)} v_0^{(2)}\ v_1^{(1)} v_1^{(2)}\ v_2^{(1)} v_2^{(2)} \ldots]$$

Example: Using generator matrix



$$\begin{pmatrix} \mathbf{g}^{(1)} = [1\,0\,1\,1] \\ \mathbf{g}^{(2)} = [1\,1\,1\,1] \end{pmatrix}$$

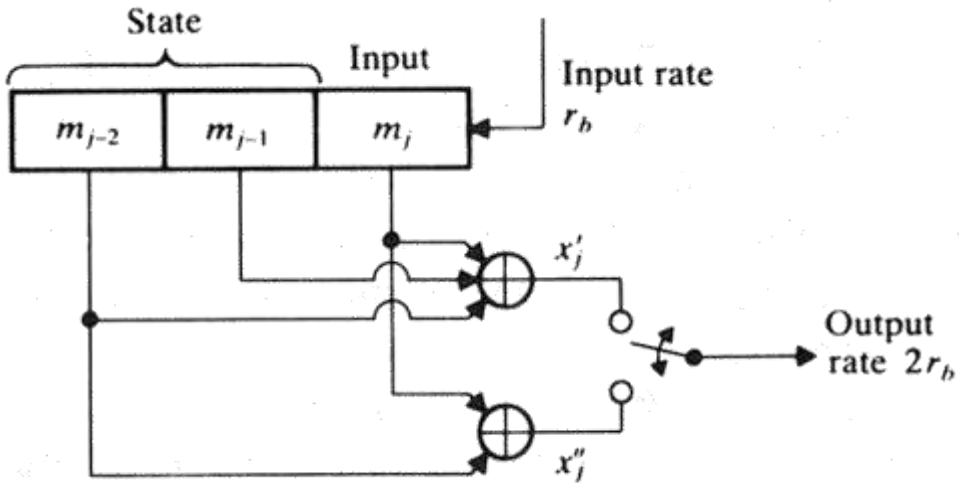
If $\mathbf{u} = (1\ 0\ 1\ 1\ 1)$, then

$$\mathbf{v} = \mathbf{uG}$$

$$= (1\ 0\ 1\ 1\ 1)\begin{bmatrix} 1\,1 & 0\,1 & 1\,1 & 1\,1 & & & \\ & 1\,1 & 0\,1 & 1\,1 & 1\,1 & & \\ & & 1\,1 & 0\,1 & 1\,1 & 1\,1 & \\ & & & 1\,1 & 0\,1 & 1\,1 & 1\,1 \end{bmatrix}$$

$$= (1\ \ 1,\ \ 0\ \ 1,\ \ 0\ \ 0,\ \ 0\ \ 1,\ \ 0\ \ 1,\ \ 0\ \ 1,\ \ 0\ \ 0,\ \ 1\ \ 1),$$

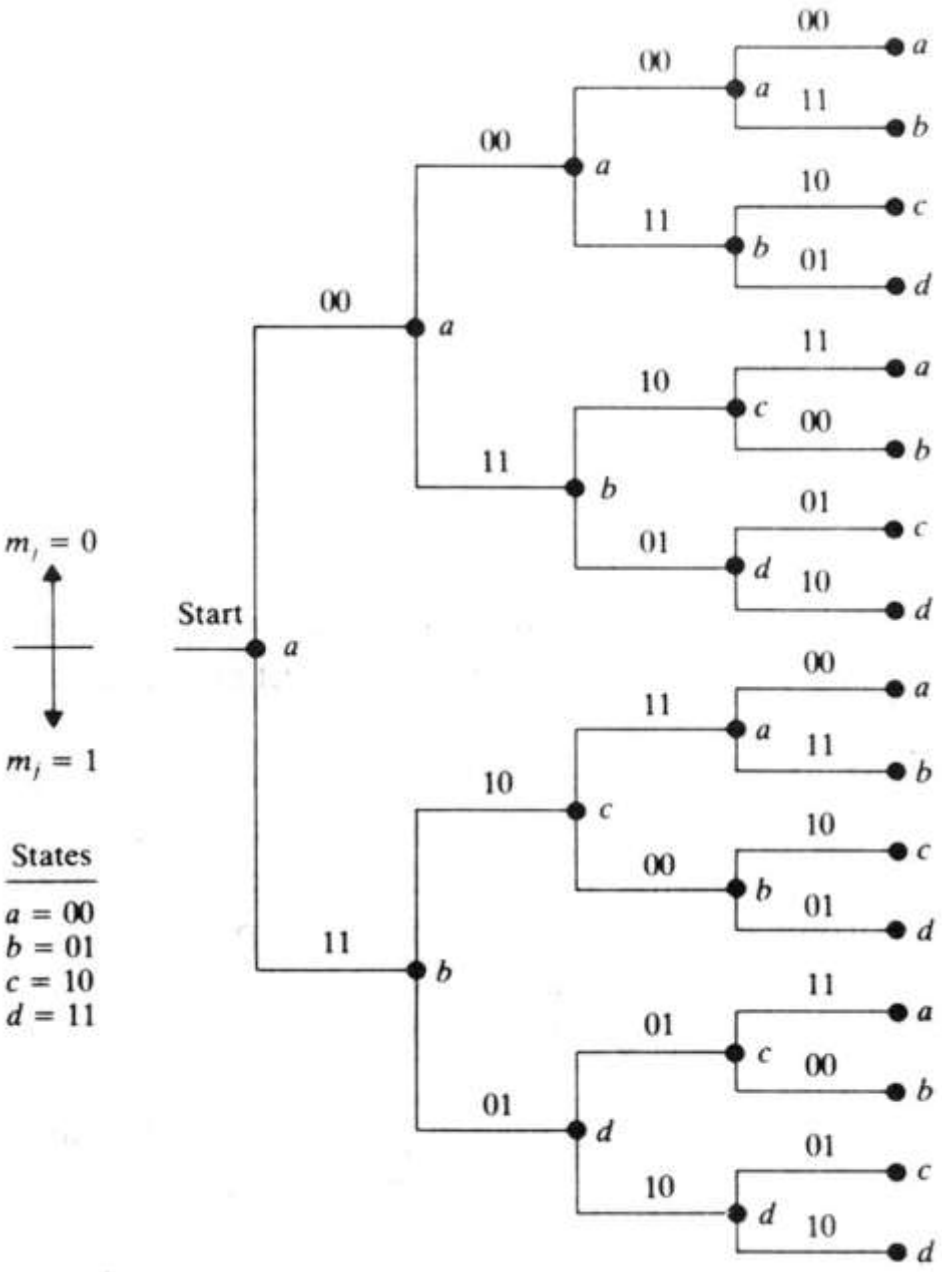
Representing convolutional codes: Code tree:



$(n,k,L) = (2,1,2)$ encoder

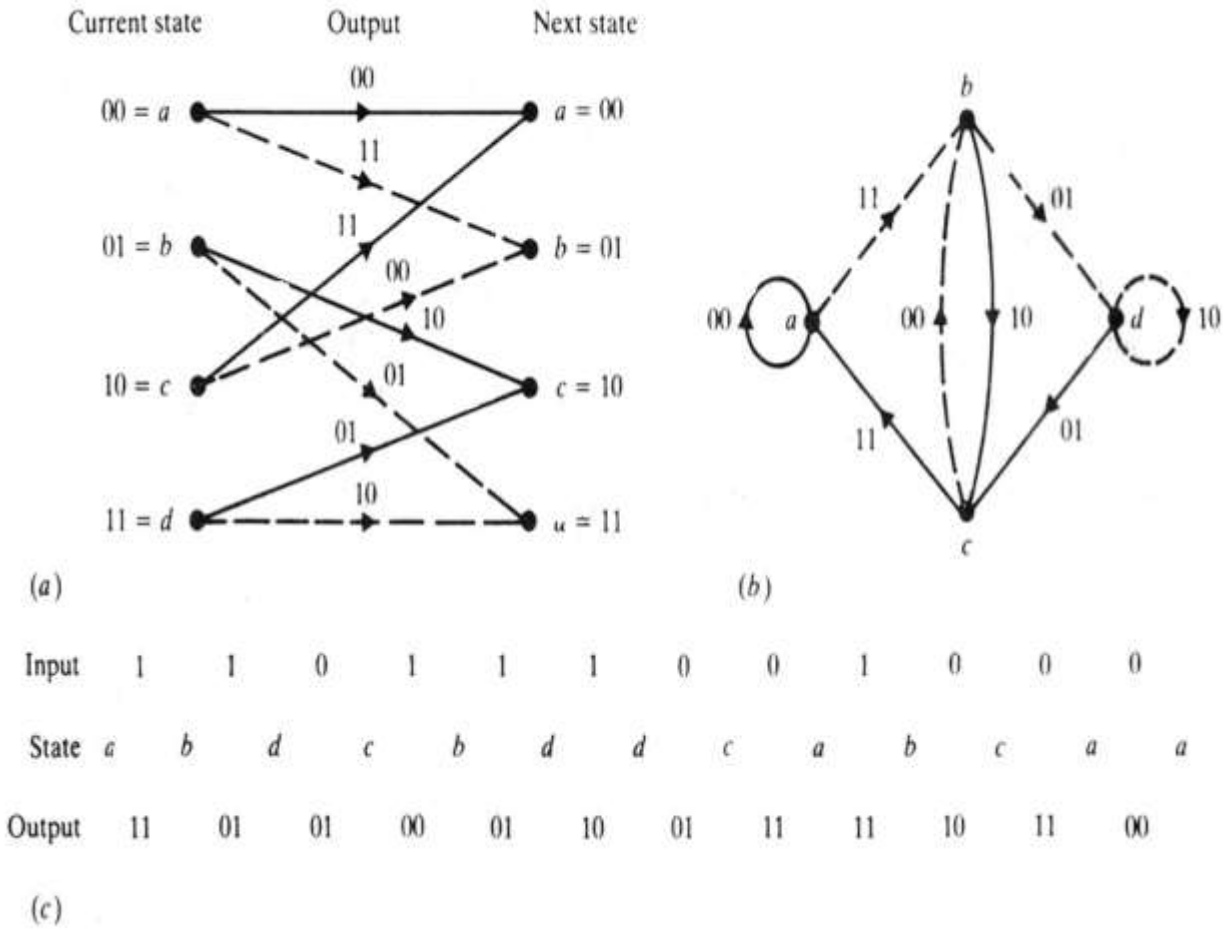
$$\begin{cases} x'_j = m_{j-2} \oplus m_{j-1} \oplus m_j \\ x''_j = m_{j-2} \oplus m_j \end{cases}$$

$$X_{out} = x'_1 x''_1 x'_2 x''_2 x'_3 x''_3 \ldots$$


Start
a
00
11
a
00
11
b
00
11
c
10
01
d
$m_j = 0$
$m_j = 1$
States
$a = 00$
$b = 01$
$c = 10$
$d = 11$

Representing convolutional codes compactly: code trellis and state diagram:

State diagram



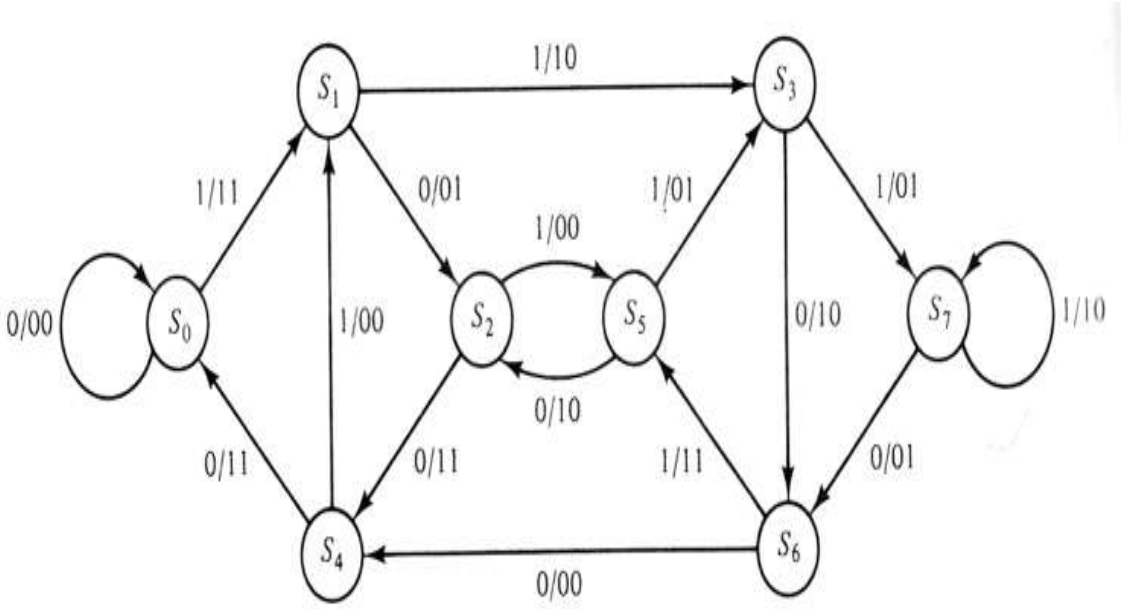
Input	1	1	0	1	1	1	0	0	1	0	0	0	
State	*a*	*b*	*d*	*c*	*b*	*d*	*d*	*c*	*a*	*b*	*c*	*a*	*a*
Output	11	01	01	00	01	10	01	11	11	10	11	00	

(c)

Inspecting state diagram: Structural properties of convolutional codes:

- Each new block of k input bits causes a transition into new state
- Hence there are $2k$ branches leaving each state

- Assuming encoder zero initial state, encoded word for any input of *k* bits can thus be obtained. For instance, below for **u**=(1 1 1 0 1), encoded word **v**=(1 1, 1 0, 0 1, 0 1, 1 1, 1 0, 1 1, 1 1) is produced:
-



- encoder state diagram for $(n,k,L)=(2,1,2)$ code
- note that the number of states is $2L+1 = 8$

Distance for some convolutional codes:

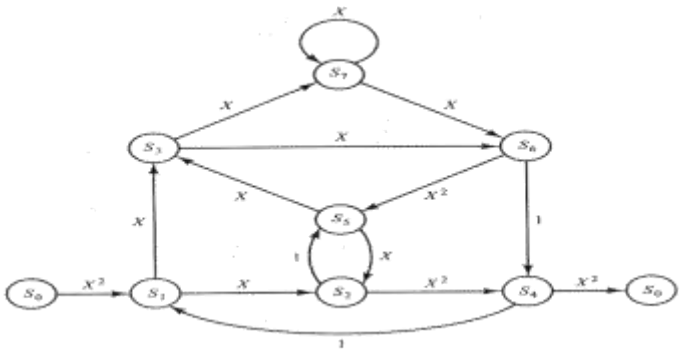
n	k	R_c	L	d_f	$R_c\, d_f/2$
4	1	1/4	3	13	1.63
3	1	1/3	3	10	1.68
2	1	1/2	3	6	1.50
			6	10	2.50
			9	12	3.00
3	2	2/3	3	7	2.33
4	3	3/4	3	8	3.00

THE VITERBI ALGORITHEM:

- Problem of optimum decoding is to find the minimum distance path from the initial state back to initial state (below from *S0* to *S0*). The minimum distance is the sum of all path metrics

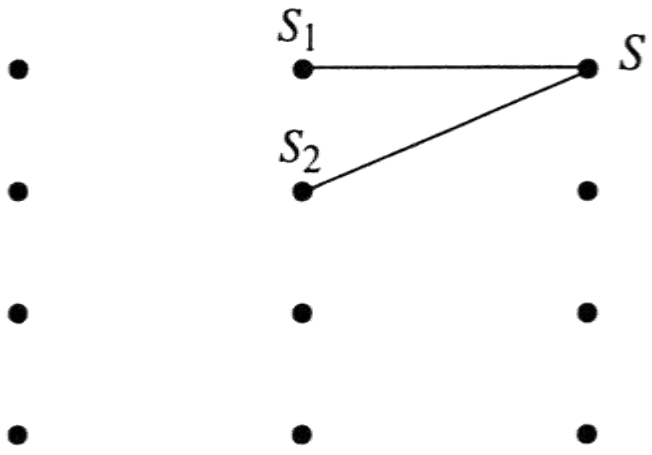
$$\ln p(\mathbf{y}, \mathbf{x}_m) = \sum_{j=0}^{\infty} \ln p(y_j \mid x_{mj})$$

- that is maximized by the correct path
- Exhaustive maximum likelihood method must search **all the paths** in phase trellis (2*k* paths emerging/ entering from 2 L+1 states for an (*n*,*k*,*L*) code)
- The Viterbi algorithm gets its efficiency via concentrating into**survivor paths** of the trellis

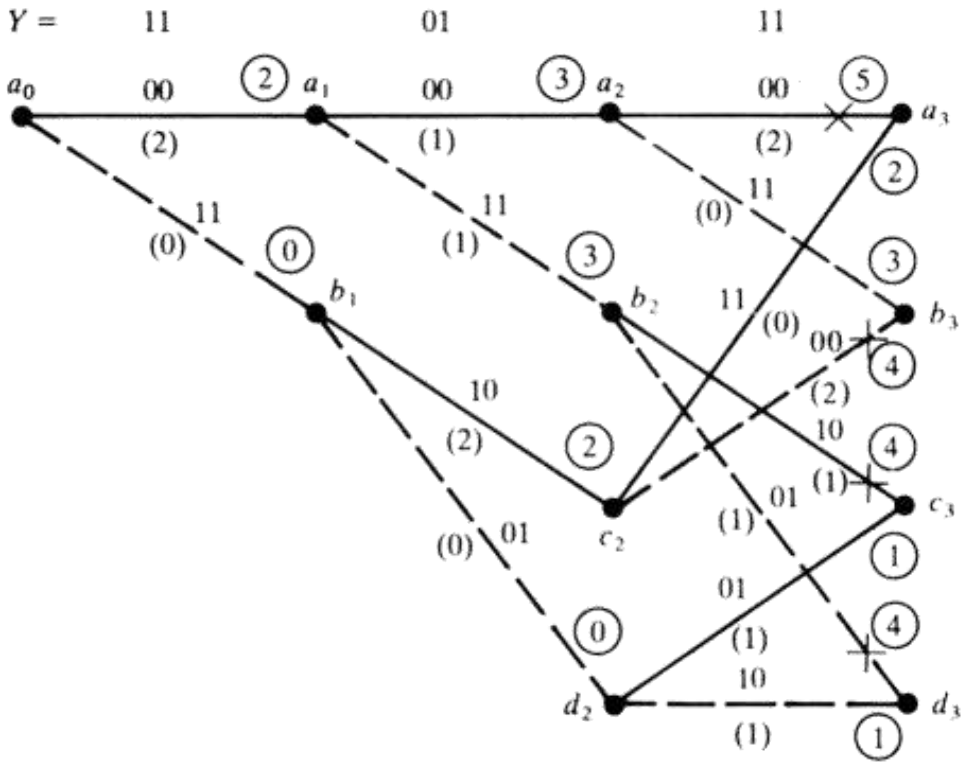


THE SURVIVOR PATH:

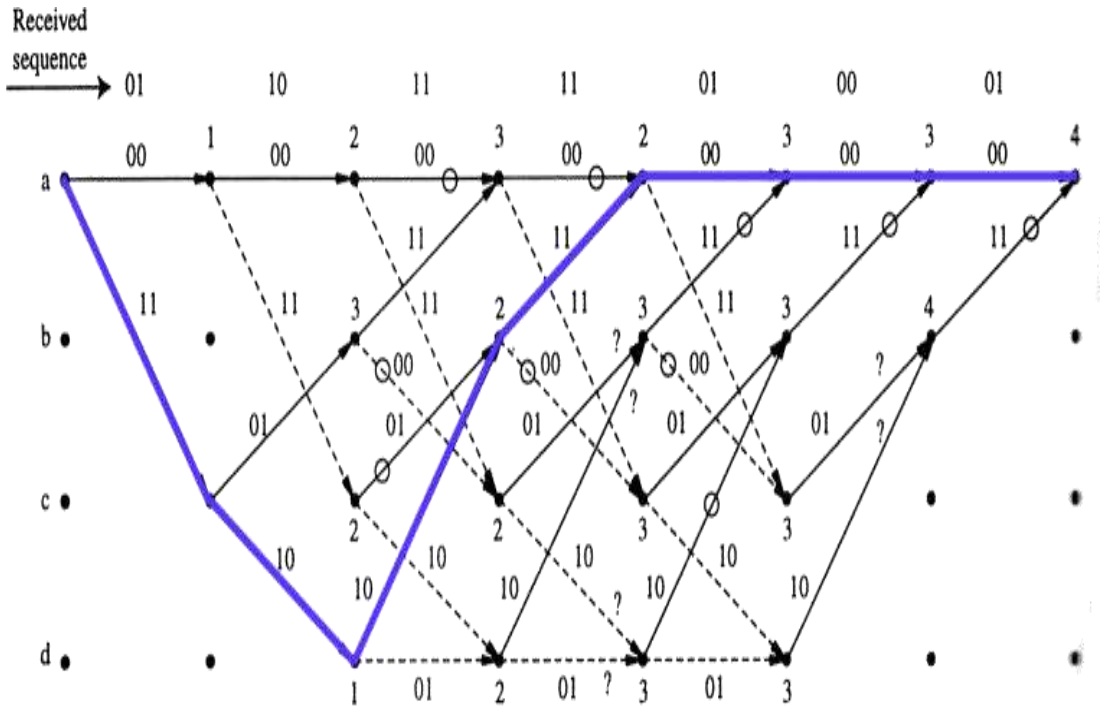
- Assume for simplicity a convolutional code with k=1, and up to $2k$ = 2 branches can enter each state in trellis diagram
- Assume optimal path passes S. Metric comparison is done by adding the metric of S into S1 and S2. At the survivor path the accumulated metric is naturally smaller (otherwise it could not be the optimum path)



- For this reason the non-survived path can be discarded -> all path alternatives need not to be considered
- Note that in principle whole transmitted sequence must be received before decision. However, in practice storing of states for input length of 5*L* is quite adequate



The maximum likelihood path:

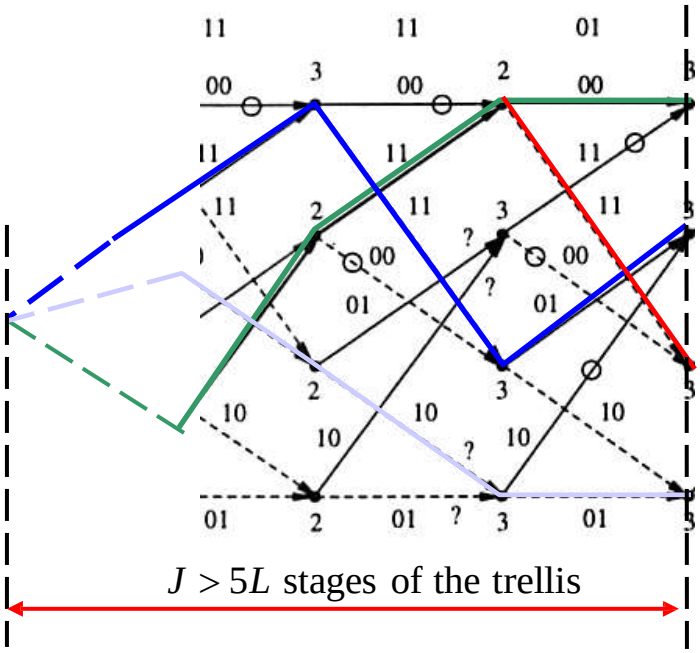


The decoded ML code sequence is 11 10 10 11 00 00 00 whose Hamming
distance to the received sequence is 4 and the respective decoded sequence is 1 1 0 0 0 0 0 (why?). Note that this is the minimum distance path.
(Black circles denote the deleted branches, dashed lines: '1' was applied)

How to end-up decoding?

- In the previous example it was assumed that the register was finally filled with zeros thus finding the minimum distance path
- In practice with long code words zeroing requires feeding of long sequence of zeros to the end of the message bits: this wastes channel capacity & introduces delay
- To avoid this *path memory truncation* is applied:
  - Trace all the surviving paths to the depth where they merge

– Figure right shows a common point at a memory depth J
– J is a random variable whose applicable magnitude shown in the figure ($5L$) has been experimentally tested for negligible error rate increase
– Note that this also introduces the delay of $5L$!



Hamming Code Example:

$$\mathbf{G} := \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \end{pmatrix}$$

- H(7,4)
- Generator matrix G: first 4-by-4 identical matrix

$$\mathbf{p} = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix}$$

- Message information vector p

- Transmission vector x
- Received vector r

and error vector e

- Parity check matrix H

$$\mathbf{r} = \mathbf{x} + \mathbf{e}_i$$

$$\mathbf{Gp} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \mathbf{x}$$

$$\mathbf{H} := \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$$

Error Correction:

- If there is no error, syndrome vector z=zeros

$$\mathbf{Hr} = \mathbf{H}(\mathbf{x} + \mathbf{e}_i) = \mathbf{Hx} + \mathbf{He}_i = 0 + \mathbf{He}_i = \mathbf{He}_i$$

- If there is one error at location 2

- New syndrome vector z is

$$\mathbf{Hr} = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} = \mathbf{z}$$

$$\mathbf{r} = \mathbf{x} + \mathbf{e}_2 = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

$$\mathbf{Hr} = \begin{pmatrix} 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = \mathbf{z}$$

Example of CRC:

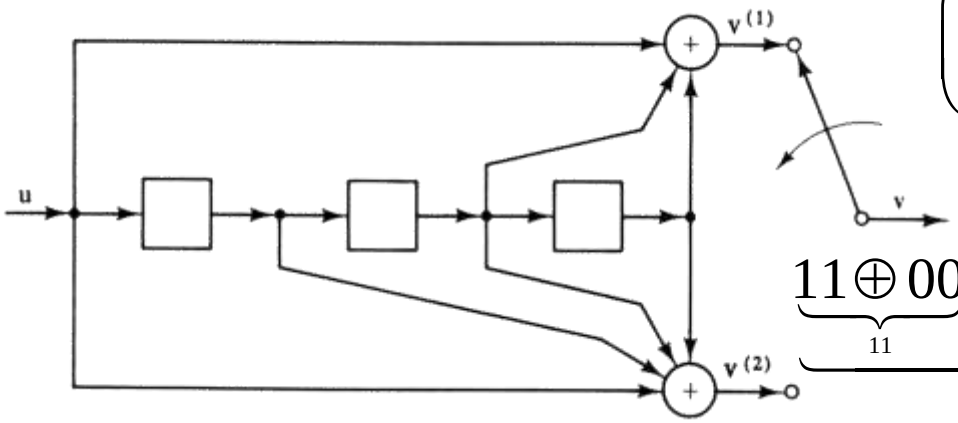
$r = 3,\ G = 1001$

$M = 110101 \Rightarrow M2^r = 110101000$

$$
\begin{array}{r}
110011 \\
1001 \overline{)\ 110101000} \\
\underline{1001} \\
01000 \\
\underline{1001} \\
0001100 \\
\underline{1001} \\
01010 \\
\underline{1001} \\
011 = \text{R (3 bits)}
\end{array}
$$

Modulo 2 Division

Example: Using generator matrix:



$$\begin{pmatrix} \mathbf{g}^{(1)} = [1\ 0\ 1\ 1] \\ \mathbf{g}^{(2)} = [1\ 1\ 1\ 1] \end{pmatrix}$$

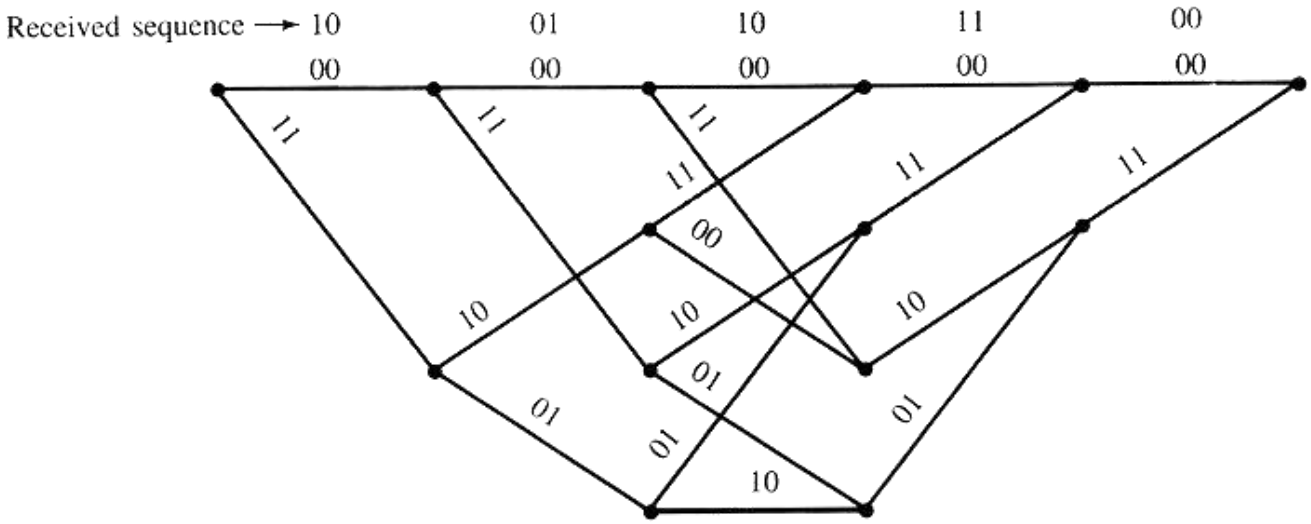
$$\underbrace{\underbrace{11 \oplus 00}_{11} \oplus \underbrace{01 \oplus 11}_{10}}_{01} = 01$$

If $\mathbf{u} = (1\ 0\ 1\ 1\ 1)$, then

$$\mathbf{v} = \mathbf{uG}$$

$$= (1\ 0\ 1\ 1\ 1)\begin{bmatrix} 1\,1 & 0\,1 & 1\,1 & 1\,1 & & & \\ & 1\,1 & 0\,1 & 1\,1 & 1\,1 & & \\ & & 1\,1 & 0\,1 & 1\,1 & 1\,1 & \\ & & & 1\,1 & 0\,1 & 1\,1 & 1\,1 \end{bmatrix}$$

$$= (1\ 1,\ 0\ 1,\ 0\ 0,\ 0\ 1,\ 0\ 1,\ 0\ 1,\ 0\ 0,\ 1\ 1),$$



correct:1+1+2+2+2=8;8 · (–0.11) = –0.88

false:1+1+0+0+0=2;2 · (–2.30) = –4.6

total path metric: –5.48

The Hamming distance between this path and the received sequence is 5. All paths (specified by the encoder input bits) and their path metrics and Hamming distances are listed below.

Received sequence: 10, 01, 10, 11, 00

Path	Code Sequence	Path Metric	Hamming Distance
0, 0, 0, 0, 0	00, 00, 00, 00, 00	−12.05	5
0, 0, 1, 0, 0	00, 00, 11, 10, 11	−14.24	6
0, 1, 0, 0, 0	00, 11, 10, 11, 00	−5.48	2
0, 1, 1, 0, 0	00, 11, 01, 01, 11	−16.43	7
1, 0, 0, 0, 0	11, 10, 11, 00, 00	−14.24	6
1, 0, 1, 0, 0	11, 10, 00, 10, 11	−16.43	7
1, 1, 0, 0, 0	11, 01, 01, 11, 00	−7.67	3
1, 1, 1, 0, 0	11, 01, 10, 01, 11	−9.86	4

Turbo Codes:

- Backgound
  - Turbo codes were proposed by Berrou and Glavieux in the 1993 International Conference in Communications.
  - Performance within 0.5 dB of the channel capacity limit for BPSK was demonstrated.
- Features of turbo codes
  - Parallel concatenated coding
  - Recursive convolutional encoders
  - Pseudo-random interleaving
  - Iterative decoding

Motivation: Performance of Turbo Codes

- Comparison:
  - Rate 1/2 Codes.
  - K=5 turbo code.
  - K=14 convolutional code.
- Plot is from:
  - L. Perez, "Turbo Codes", chapter 8 of Trellis Coding by C. Schlegel. IEEE Press, 1997

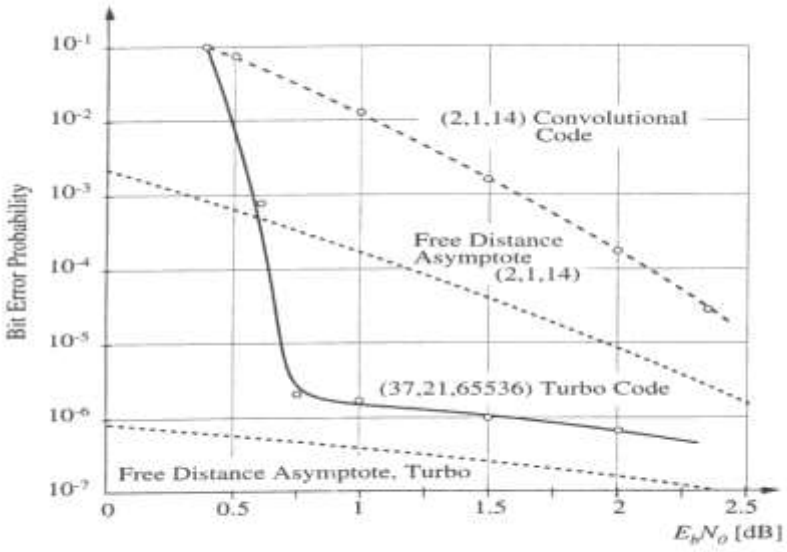


Figure 8.1 Simulated performance of the original Turbo code and the code's free-distance asymptote and the simulated performance of the (2, 1, 14) MFD convolutional code.

Pseudo-random Interleaving:

- The coding dilemma:
  - Shannon showed that large block-length random codes achieve channel capacity.
  - However, codes must have structure that permits decoding with reasonable complexity.
  - Codes with structure don"t perform as well as random codes.
  - "Almost all codes are good, except those that we can think of."

- Solution:
  - Make the code appear random, while maintaining enough structure to permit decoding.
  - This is the purpose of the pseudo-random interleaver.
  - Turbo codes possess random-like properties.
  - However, since the interleaving pattern is known, decoding is possible.

Why Interleaving and Recursive Encoding?

- In a coded systems:
  - Performance is dominated by low weight code words.
- A "good" code:
  - will produce low weight outputs with very low probability.
- An RSC code:
  - Produces low weight outputs with fairly low probability.
  - However, some inputs still cause low weight outputs.
- Because of the interleaver:
  - The probability that both encoders have inputs that cause low weight outputs is very low.
  - Therefore the parallel concatenation of both encoders will produce a "good" code.

Iterative Decoding:

- There is one decoder for each elementary encoder.
- Each decoder estimates the *a posteriori probability* (APP) of each data bit.
- The APP"s are used as *a priori* information by the other decoder.
- Decoding continues for a set number of iterations.